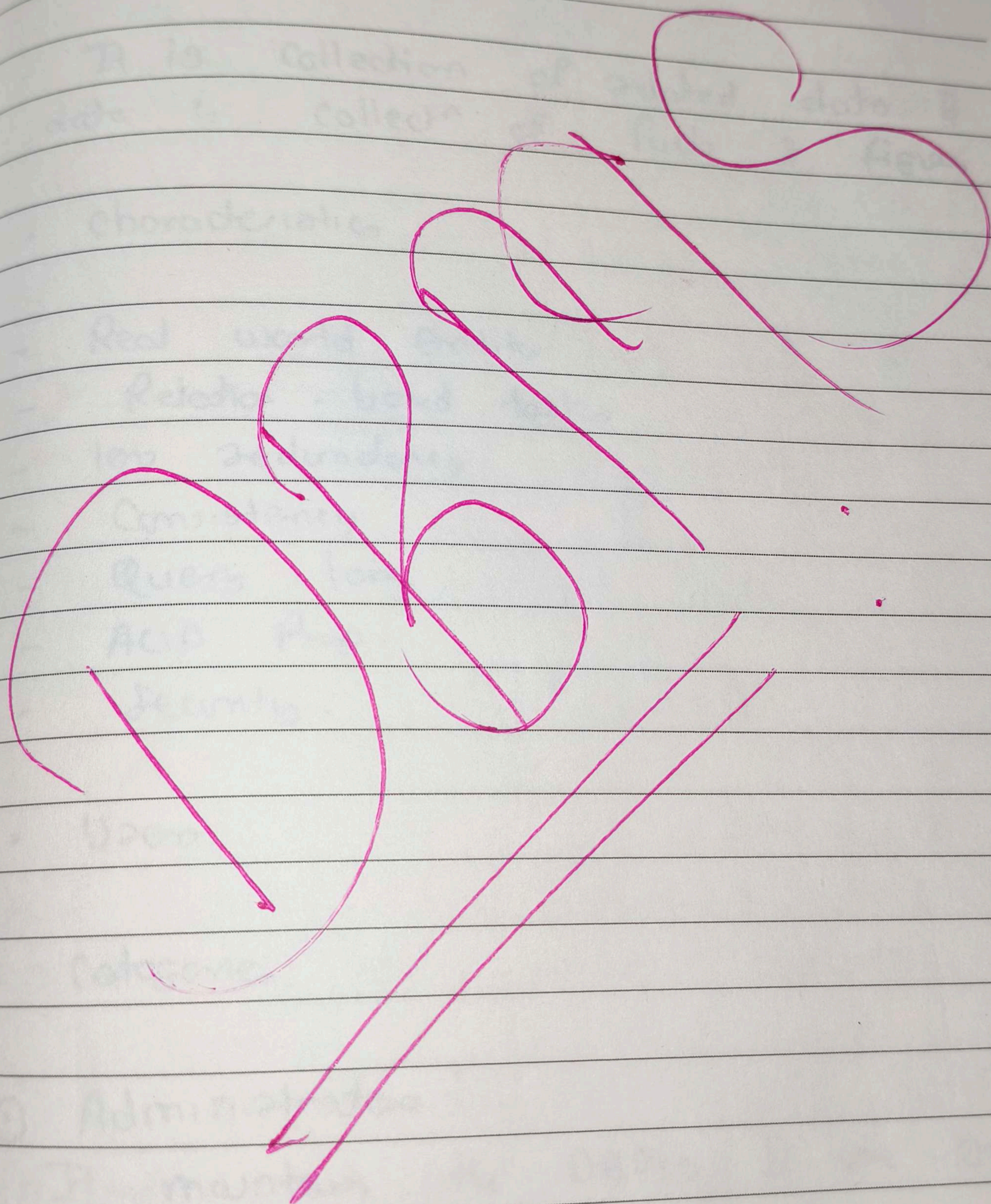


Date : _____



• Database

It is collection of related data & data is collectⁿ of facts & figures

• Characteristics

- Real world Entity
- Relation - based tables
- less redundancy
- Consistency
- Query lang.
- ACID Prop.
- Security.

• Users

- Categories

① Administrator :

- It maintain the DBMS & are responsible for administering the db.
- They are responsible to look after its usage & by whom it should be used

Date : _____

• Designer :

- People who actually work on designing part.

• End user :

- user.

• 3 - tier Architecture.

Presentation Tier

Appⁿ tier

Database tier.

① Database tier :

- Query processing lang.
- app that define data & constraint

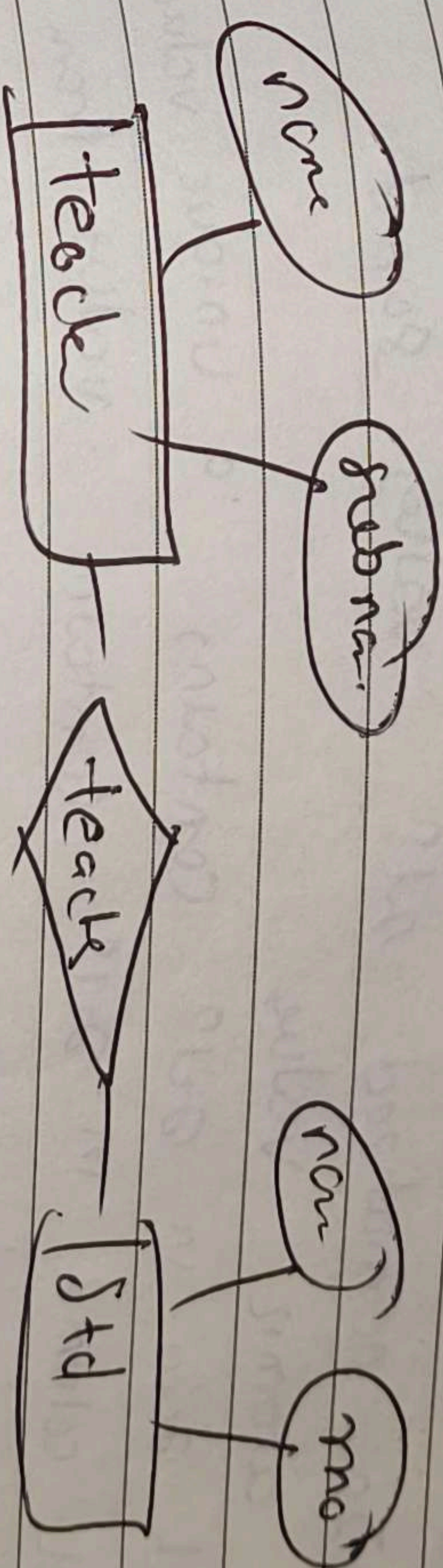
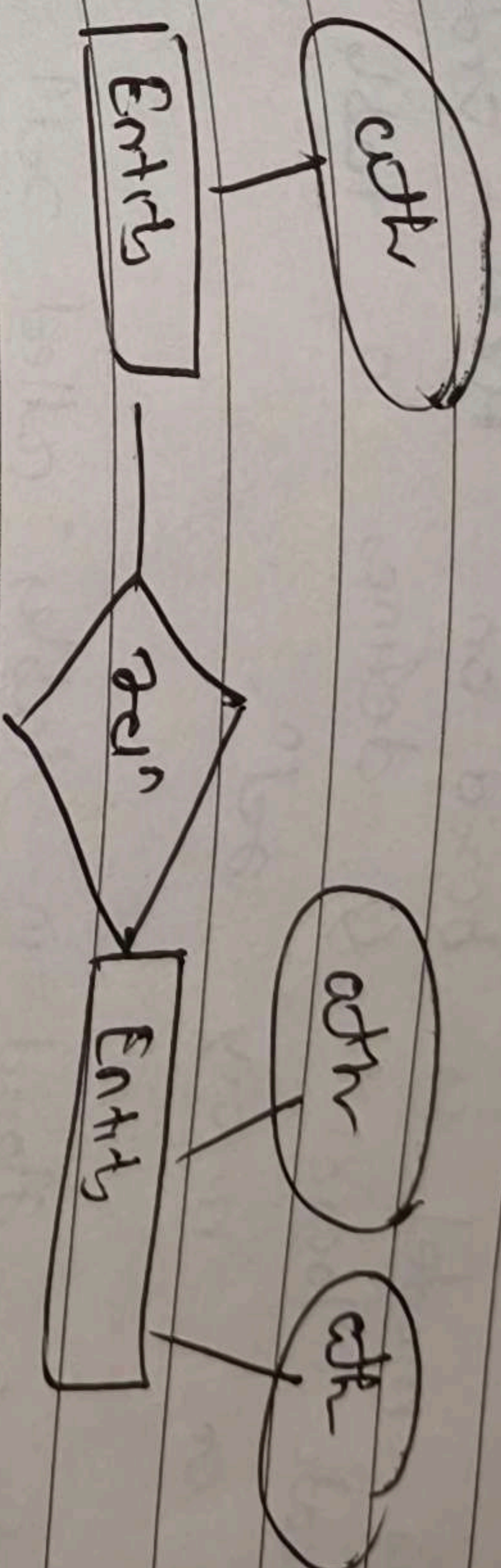
② Appⁿ tier :

- In this process that access the db

③ User tier. (Presentation)

Date : _____

• ER (Entity Relation Model)



• Entity

- An entity in an ER model is real world entity having prop. called attribute

• Relationship

- The logical connectⁿ among entities
- Mapping Cardinality :
 - one - to - one
 - one - to - many
 - many - to - one
 - many - to - many

Date : _____

• Relation Model

- This model is based on first-order predicate logic & defines a table or an n-ary relⁿ.

- Data is stored in tables called rel^s.

- Rel^s can be normalized

- In normalized rel^s, values stored are atomic values.

- Each row in rel^s contains a unique value

- Each column in rel^s contains values from a same domain

• Database Schema

- It is the skeleton structure that represents the logical view of the entire db.

- It defines how the data is organized & how the rel^s among them are associated

Date : _____

• Physical DB Schema

- It is actual stores of data like files, indices, etc.

• Logical DB Schema

- This schema defines all the logical constraints that need to be applied on data stored

- It defines tables, views & integrity constraints.

• Database Instance

- It is state of operational db with data at any given time.

- It contains a snapshot of db.

- Db instances tend to change with time.

• Data Independence

① Logical Data Independence

Date : _____

- It is data about db, that is, it stores info. about how data is managed inside

① Physical Data Independence

- It is the power to change the physical data without impacting the schema or logical data.

• ER Model

- It is Conceptual model for designing a db.

- It represents the logical structure of db, including entities, attr & rel.

• Symbols

Rectangle - Entity

Ellipse - attributes

Diamond - Relⁿ

Line - attr type

Double ellipse - Mult. value attr

Double rectangles - Weak Entity

Date : _____

• Entity - An obj that is stored in data.
• Entity set - Collection of entity set.

• Types :

① Strong Entity : A key attr that can uniquely identify each instance of entity.

② Weak Entity : Cannot be uniquely identified by its own attr alone.

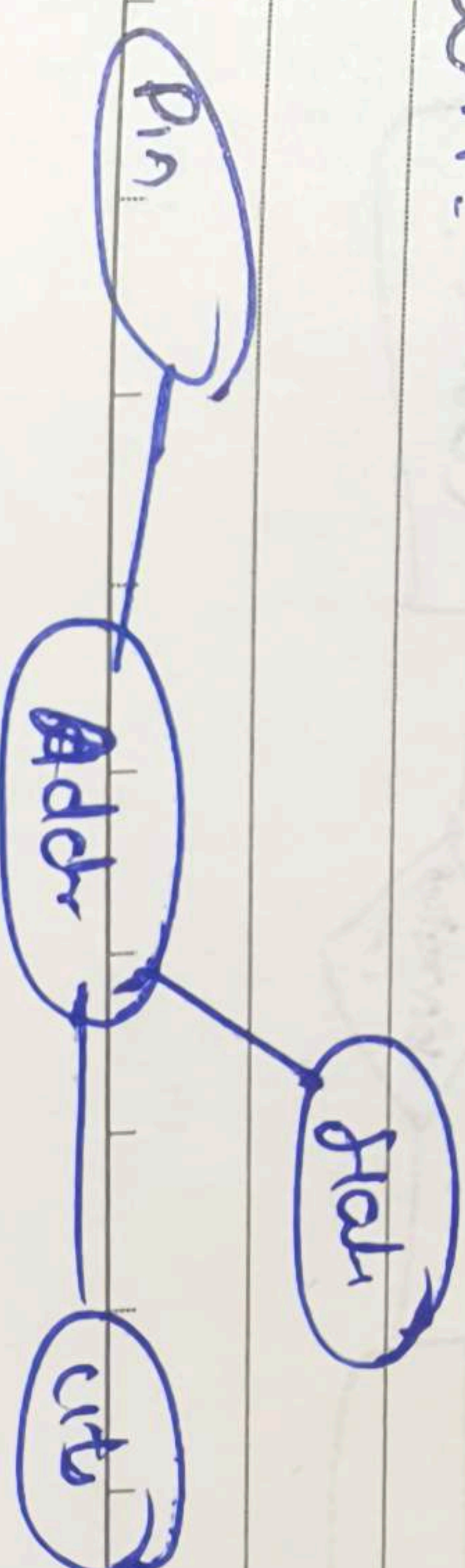
• Attributes - Properties that describe the entities

• Types

① Key attr - Uniquely identifies - Pno

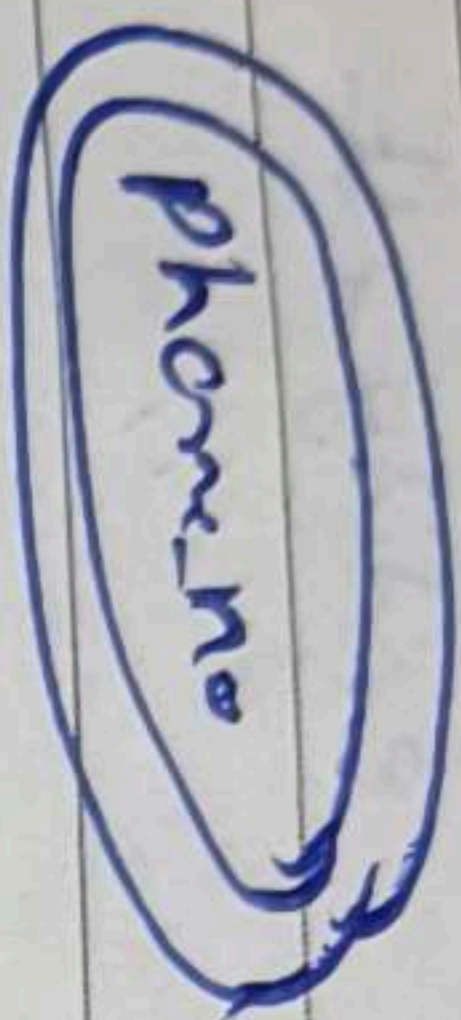
② Composite attr.

- An attr. composed of many other attr.

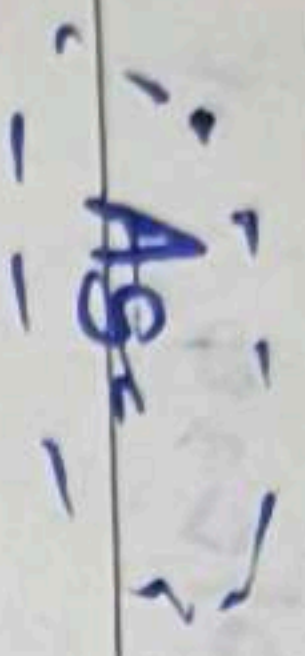


Date : _____

(iii) Multivalued attr. - more than one value for given entity



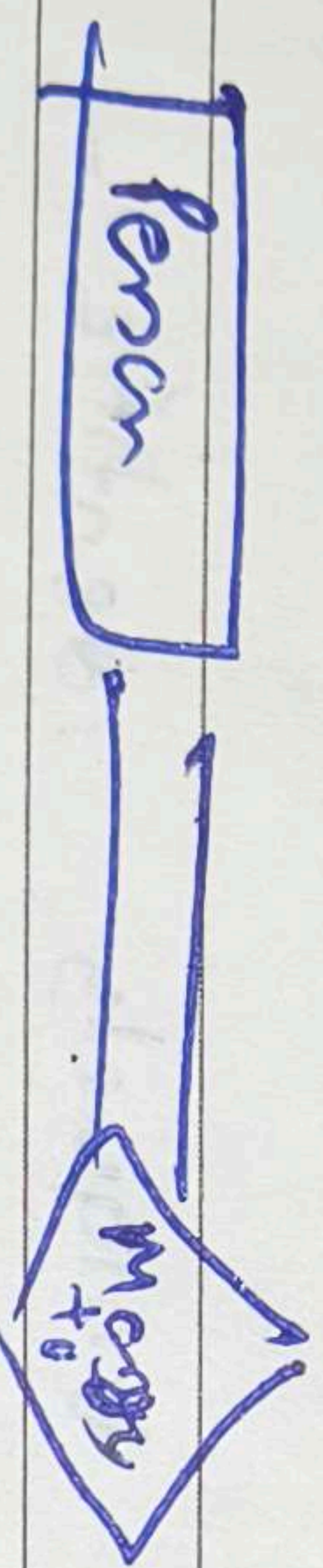
(iv) Derived attr. - attr. derived from other attr. of entity type



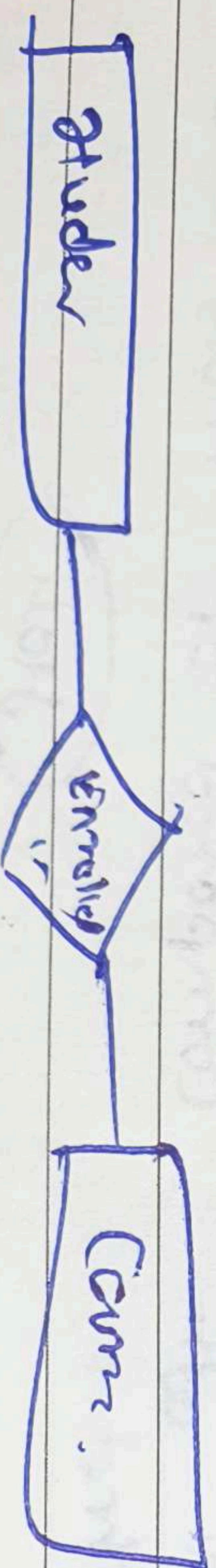
• Relationship type / set
- association betⁿ entity type

• Degree.

(i) Unary set - only one entity participating in set



(ii) Binary set - two entities



Date : _____

(iii) Ternary set : three entities

(iv) N-ary set : n entities

• Cardinality in ER Model

- The max. no. of time an entity participates in set

(i) one - to - one

(ii) one - to - Many

(iii) Many - to - one

(iv) Many - to - Many

• Participation Constraints

(i) Total Participation : Each entity in entity set must participate in set

(ii) Partial Participation : The entity may or may not participate

Date : _____

• How to draw an ER Diagram?

- (i) Identify Entities
- (ii) Identify Rel's
- (iii) Add attributes
- (iv) Define Primary keys
- (v) Remove Redundancy
- (vi) Review for clarity

• Generalization

- Where the Generalized entities contain the prop. of all generalized entities
- Bottom up method

• Specialization

- In Specialization, a grp of entities is divided into sub grps based on their characteristics
- Top down method

• Codd's 12 Rules (Google)

Date : _____

• Relational Data Model.

It is the primary data model, which is used widely around the world for data storage & processing.

• Concepts

(i) Table

(ii) Tuple : A single row of table which contains a single record for that rel.

(iii) Rel Instance : A finite set of tuples in a rel. db system represents rel instance

(iv) Rel Key : Each row has one or more attr, known as rel key

(v) Rel Schema : It describes the rel name, attr. & names.

Date : _____

(vi) Attr. domain : Every attr. has some predefined value set.

- Constraints

- It define is rules that define limitat^r & acqui for data within
 - It , ensuring data integrity, accuracy & consistency

① Domain Constraint.

- Each domain must contain atomic value which means composite & multi value are not allowed

(i) Key Constraints - Unique

(ii) Entity Integrity Constraints

- It says no primary key can take

Null value, since cons^t th

primary key.

Date : _____

(iii) Referential Integrity Constraints

- It is specified betⁿ two attr^s or table and used to maintain the consistency means th tuples "

- Relⁿ Algebra (Google)

- ER Model to Relⁿ Model (Google)

- SQL

- Normalization

- Normalizⁿ used in db design to reduce redundancy & improve data integrity by organizing data into table & ensuring proper relⁿ.

- Normal Form (Google)

Date : _____

• JOINS (JOL)

• Storage System

- Manage how data is stored & retrieved

- Databases are stored in file formats which contains records.

① Primary storage / memory

- It is directly accessible by CPU, meaning that it doesn't require any other devices to read from it

- eg. Cache memory

② Secondary memory

- It is used to store data for future use or backup

- eg. hard disk, flash drives, magnetic disks, optical disk (CD, DVD)

Date : _____

③ Tertiary Storage

- It is used to store huge volumes of data.

- eg. Optical storage (CD, DVD), Tape storage

• Memory hierarchy

- It organizes storage into different levels based on speed, cost & capacity.

• RAID (Google)

• File Organization

- It defines how file records are mapped onto disk blocks

① Heap file Organization

- It is simple method where records are stored in blocks typically at the end of file, without any specific ordering

Date :

- (i) Sequential file Organization
- records stored in specific order
 - magnetic tape

- (ii) Hash file Organization
- It uses a hash fn to map records to specific storage location, allowing for fast record retrieval based on key.

- (iii) Clustered file Organization
- Stores multiple related records from diffⁿ tables within the same physical storage block

• File operations

- open, write, close, locate, read

• Indexing

- It is used in db to optimize data retrieval operation.

Date :

- (i) Primary index :

- It is defined on an ordered data file

- The data file is ordered on a key field

- (ii) Secondary Index

- It may be generated from a field which is a candidate key & has a unique value in every record, or non-key with duplicate values

- (iii) Clustering Index

- It is defined on ordered data file

- The data file is ordered on a non-key field

• Ordered Indexing types

- (i) Dense Index - It contains an index record for every search key value in file

- (ii) Sparse Index - It contains few index entries for some records

Date: _____

• B+ Tree (Google + Yt)

• Hashing

- It is a tech. to quickly locate a data record in db irrespective of size of db.

• Hash table

- It is an array or data structure & its size is determined by total volume of data

• Bucket : It is a memory location in hash table that stores the data record.

• Hash fn : It is a mathematical fn that maps a record's key to a specific index in hash table, facilitating fast data retrieval

Date: _____

• Static hashing : fixed size
• Dynamic - : allows table to grow dynamically.

• Bucket Overflow

• Operations :

- Insertions, Update, Deletion & Inaction

• Transactions :

- It is defined as grp of tasks.
- A single task is mini. processing unit which cannot be divided further

• ACID Properties (Google)

• Status of Transaction (Google)

• Concurrency Control (Google)

• Deadlock (Google)

• Data backup & recovery (Google)

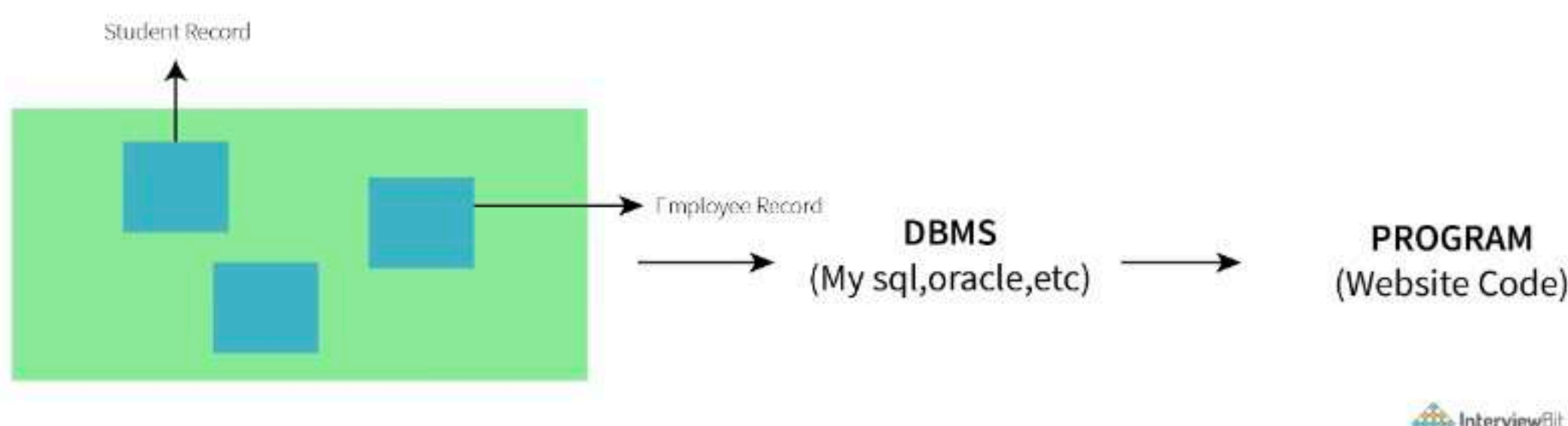


DBMS Basic Interview Questions

1. What is DBMS and what is its utility? Explain RDBMS with examples.

DBMS stands for [Database Management System](#), is a set of applications or programs that enable users to create and maintain a database. DBMS provides a tool or an interface for performing various operations such as inserting, deleting, updating, etc. into a database. It is software that enables the storage of data more compactly and securely as compared to a file-based system. A DBMS system helps a user to overcome problems like data inconsistency, data redundancy, etc. in a database and makes it more convenient and organized to use it. Check this [DBMS Tutorial by Scaler Topics](#).

Examples of popular DBMS systems are file systems, XML, Windows Registry, etc.



RDBMS stands for **Relational Database Management System** and was introduced in the 1970s to access and store data more efficiently than DBMS. RDBMS stores data in the form of tables as compared to DBMS which stores data as files. Storing data as rows and columns makes it easier to locate specific values in the database and makes it more efficient as compared to DBMS.

Examples of popular RDBMS systems are MySQL, Oracle DB, etc.

[Learn More](#)

2. What is a Database?

A Database is an organized, consistent, and logical collection of data that can easily be updated, accessed, and managed. Database mostly contains sets of tables or objects (anything created using create command is a database object) which consist of records and fields. A tuple or a row represents a single entry in a table. An attribute or a column represents the basic units of data storage, which contain information about a particular aspect of the table. DBMS extracts data from a database in the form of queries given by the user.

3. Mention the issues with traditional file-based systems that make DBMS a better choice?

The absence of indexing in a traditional file-based system leaves us with the only option of scanning the full page and hence making the access of content tedious and super slow. The other issue is redundancy and inconsistency as files have many duplicate and redundant data and changing one of them makes all of them inconsistent. Accessing data is harder in traditional file-based systems because data is unorganized in them.

Another issue is the lack of concurrency control, which leads to one operation locking the entire page, as compared to DBMS where multiple operations can work on a single file simultaneously.

Integrity check, data isolation, atomicity, security, etc. are some other issues with traditional file-based systems for which DBMSs have provided some good solutions.

4. Explain a few advantages of a DBMS.

Following are the few [advantages of using a DBMS](#).



DBMS



API

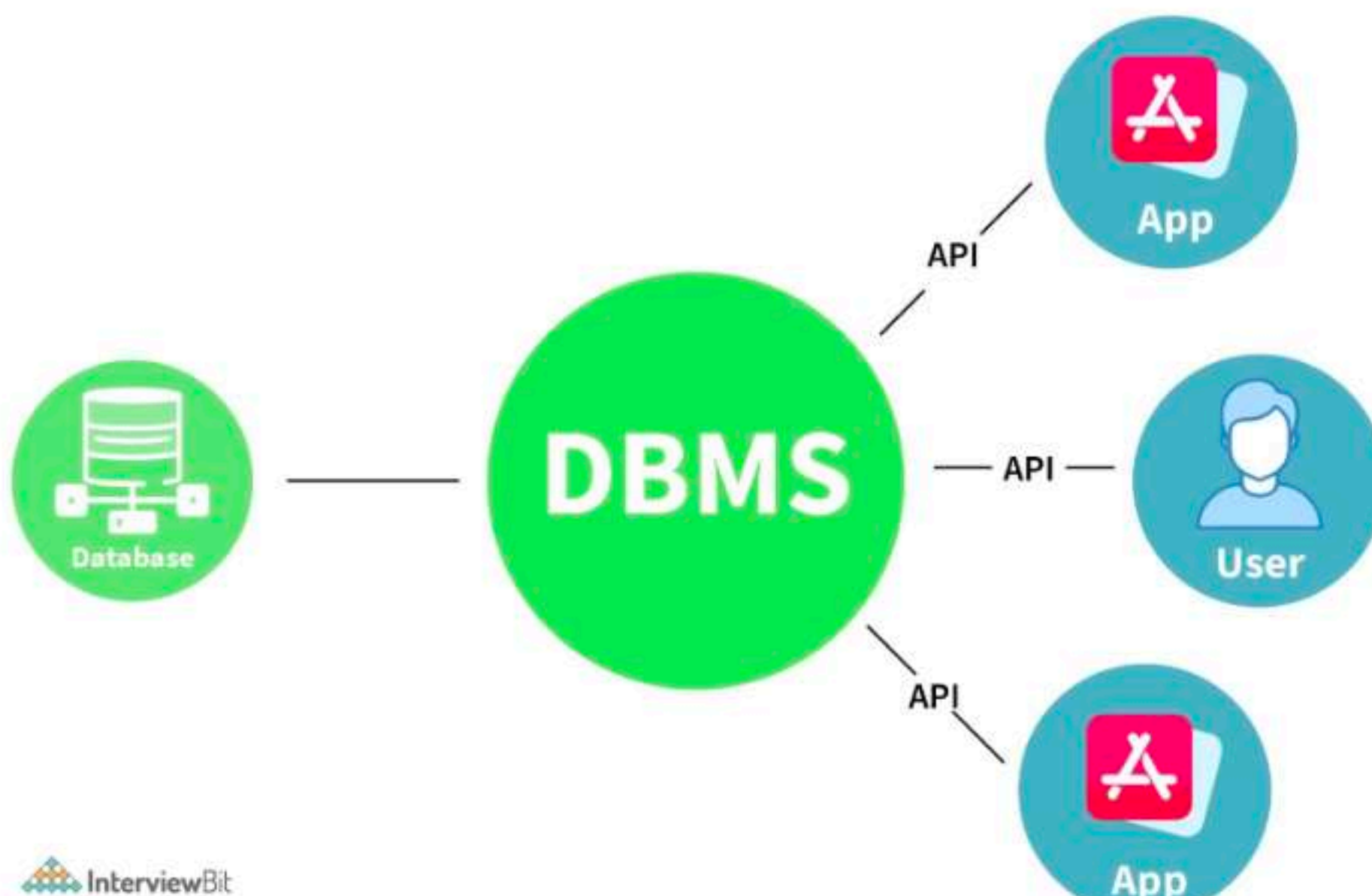


Get Ready with Free Mock Coding Interview

Integrity check, data isolation, atomicity, security, etc. are some other issues with traditional file-based systems for which DBMSs have provided some good solutions.

4. Explain a few advantages of a DBMS.

Following are the few [advantages of using a DBMS](#).



- **Data Sharing:** Data from a single database can be simultaneously shared by multiple users. Such sharing also enables end-users to react to changes quickly in the database environment.
- **Integrity constraints:** The existence of such constraints allows storing of data in an organized and refined manner.
- **Controlling redundancy in a database:** Eliminates redundancy in a database by providing a mechanism that integrates all the data in a single database.
- **Data Independence:** This allows changing the data structure without altering the composition of any of the executing application programs.
- **Provides backup and recovery facility:** It can be configured to automatically create the backup of the data and restore the data in the database whenever required.
- **Data Security:** DBMS provides the necessary tools to make the storage and transfer of data more reliable and secure. Authentication (the process of giving restricted access to a user) and encryption (encrypting sensitive data such as OTP, credit card information, etc.) are some popular tools used to secure data in a DBMS.

5. Explain different languages present in DBMS.

Following are various languages present in DBMS:

- **DDL(Data Definition Language):** It contains commands which are required to define the database. E.g., CREATE, ALTER, DROP, TRUNCATE, RENAME, etc.
- **DML(Data Manipulation Language):** It contains commands which are required to manipulate the data present in the database. E.g., SELECT, UPDATE, INSERT, DELETE, etc.
- **DCL(Data Control Language):** It contains commands which are required to deal with the user permissions and controls of the database system. E.g., GRANT and REVOKE.
- **TCL(Transaction Control Language):** It contains commands which are required to deal with the transaction of the database. E.g., COMMIT, ROLLBACK, and SAVEPOINT.

6. What is meant by ACID properties in DBMS?

ACID stands for Atomicity, Consistency, Isolation, and Durability in a DBMS these are those properties that ensure a safe and secure way of sharing data among multiple users.

DATABASE TRANSACTIONS

A

Atomic

All changes to the data must be performed successfully or not at all

C

Consistent

Data must be in a consistent state before and after the transaction

I

Isolated

No other process can change the data while the transaction is running

D

Durable

The changes made by a transaction must persist

- **Atomicity:** This property reflects the concept of either executing the whole query or executing nothing at all, which implies that if an update occurs in a database then that update should either be reflected in the whole database or should not be reflected at all.

DATABASE TRANSACTIONS

- A

Atomic
All changes to the data must be performed successfully or not at all
- C

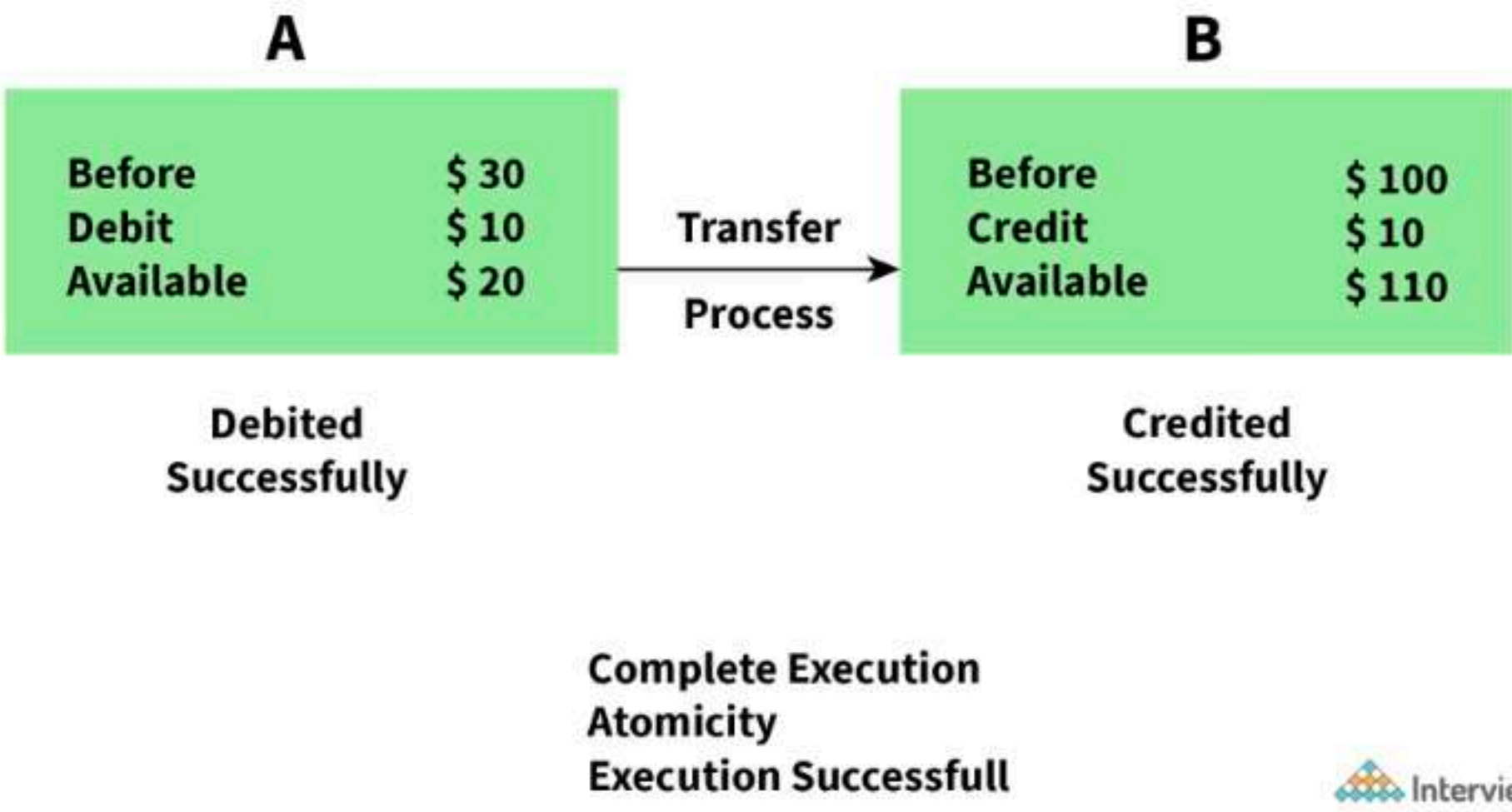
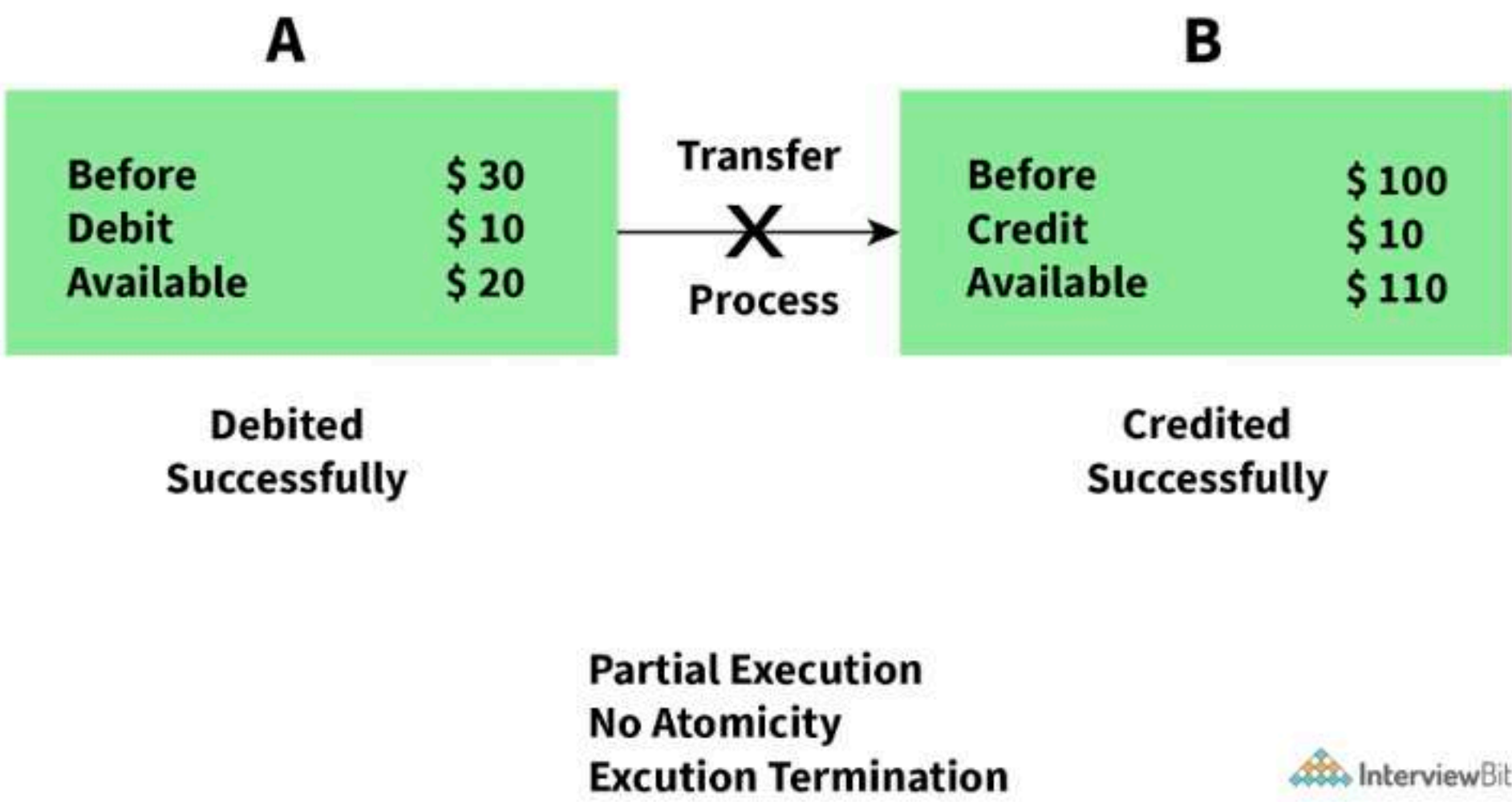
Consistent
Data must be in a consistent state before and after the transaction
- I

Isolated
No other process can change the data while the transaction is running
- D

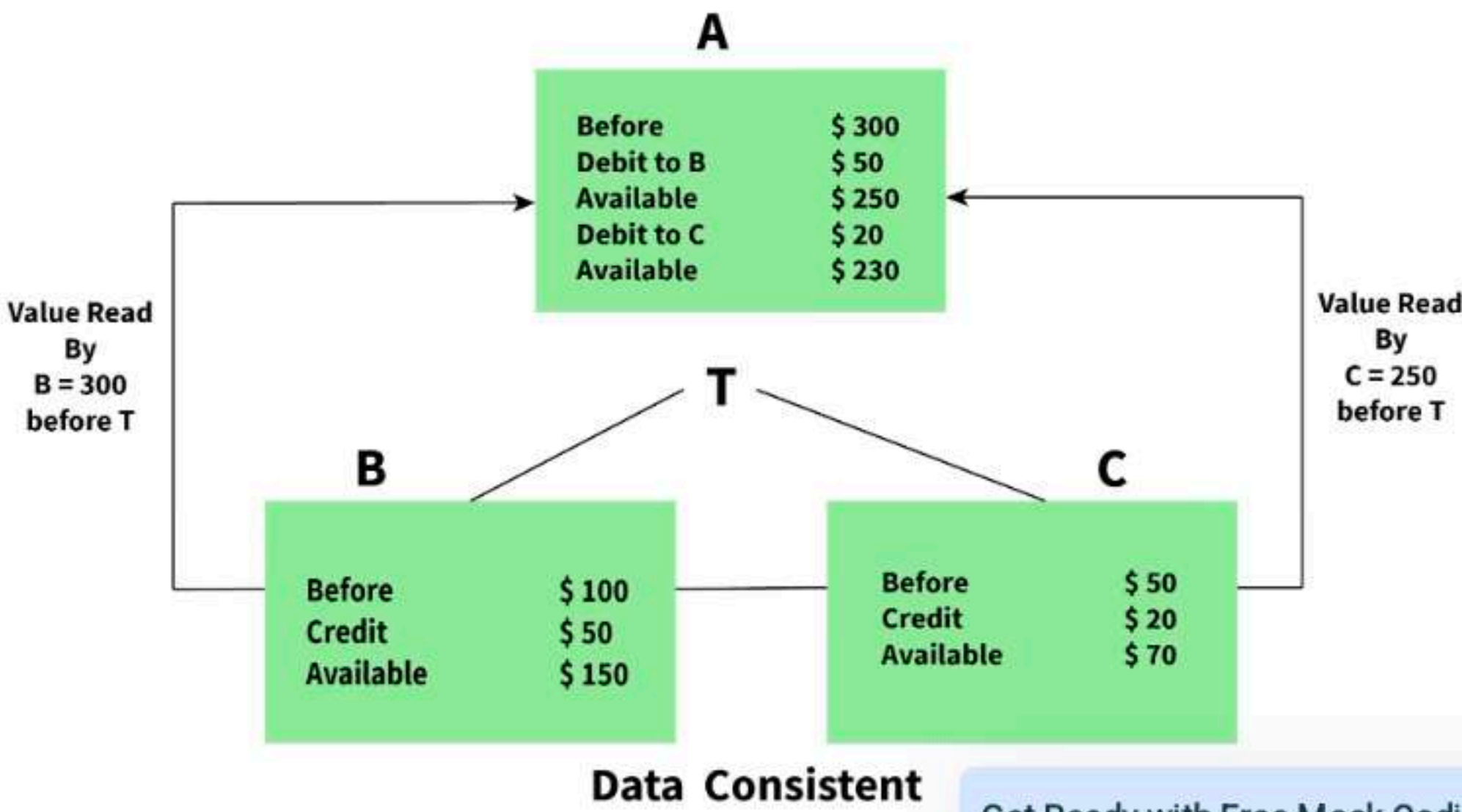
Durable
The changes made by a transaction must persist



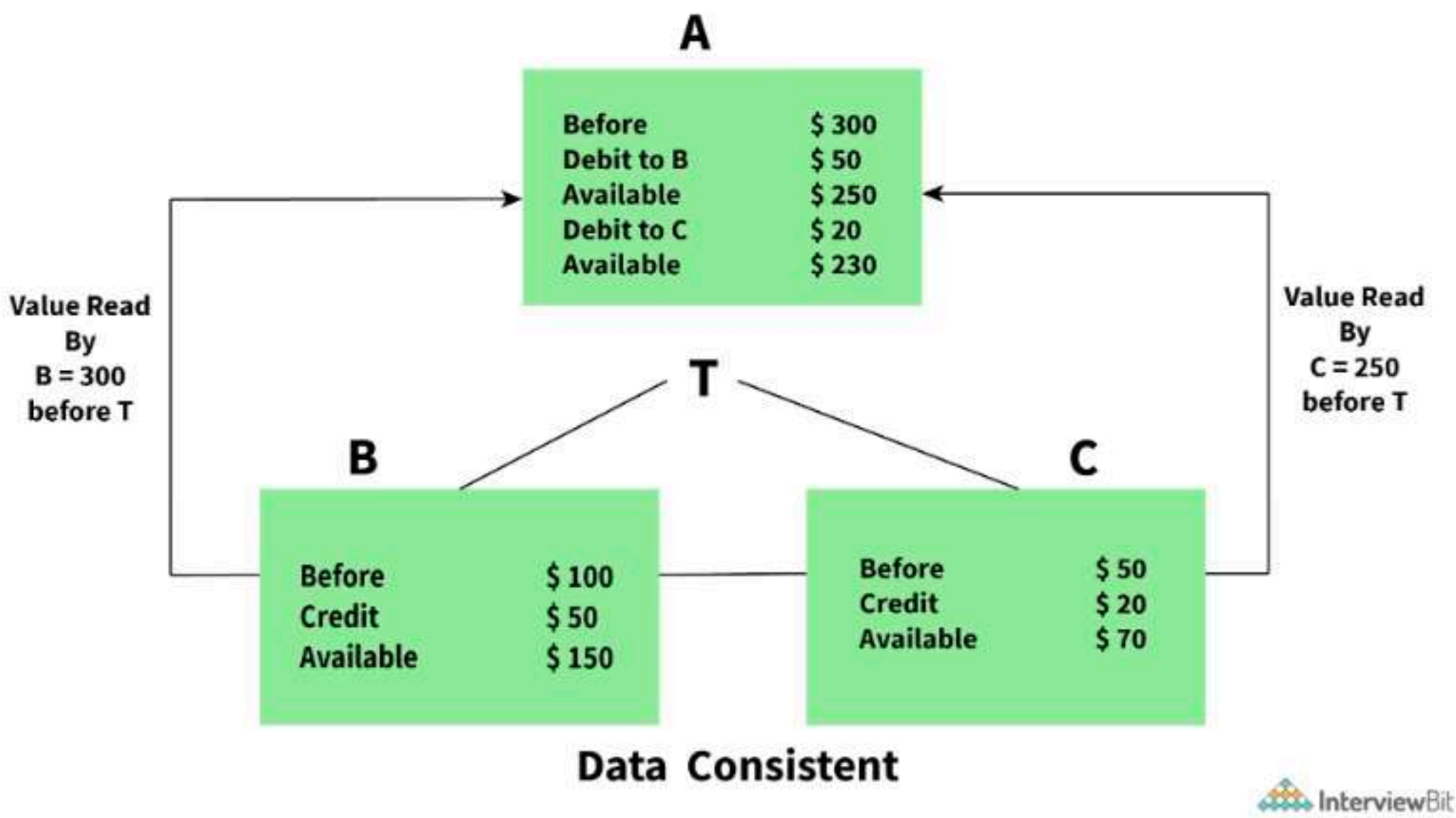
• **Atomicity:** This property reflects the concept of either executing the whole query or executing nothing at all, which implies that if an update occurs in a database then that update should either be reflected in the whole database or should not be reflected at all.



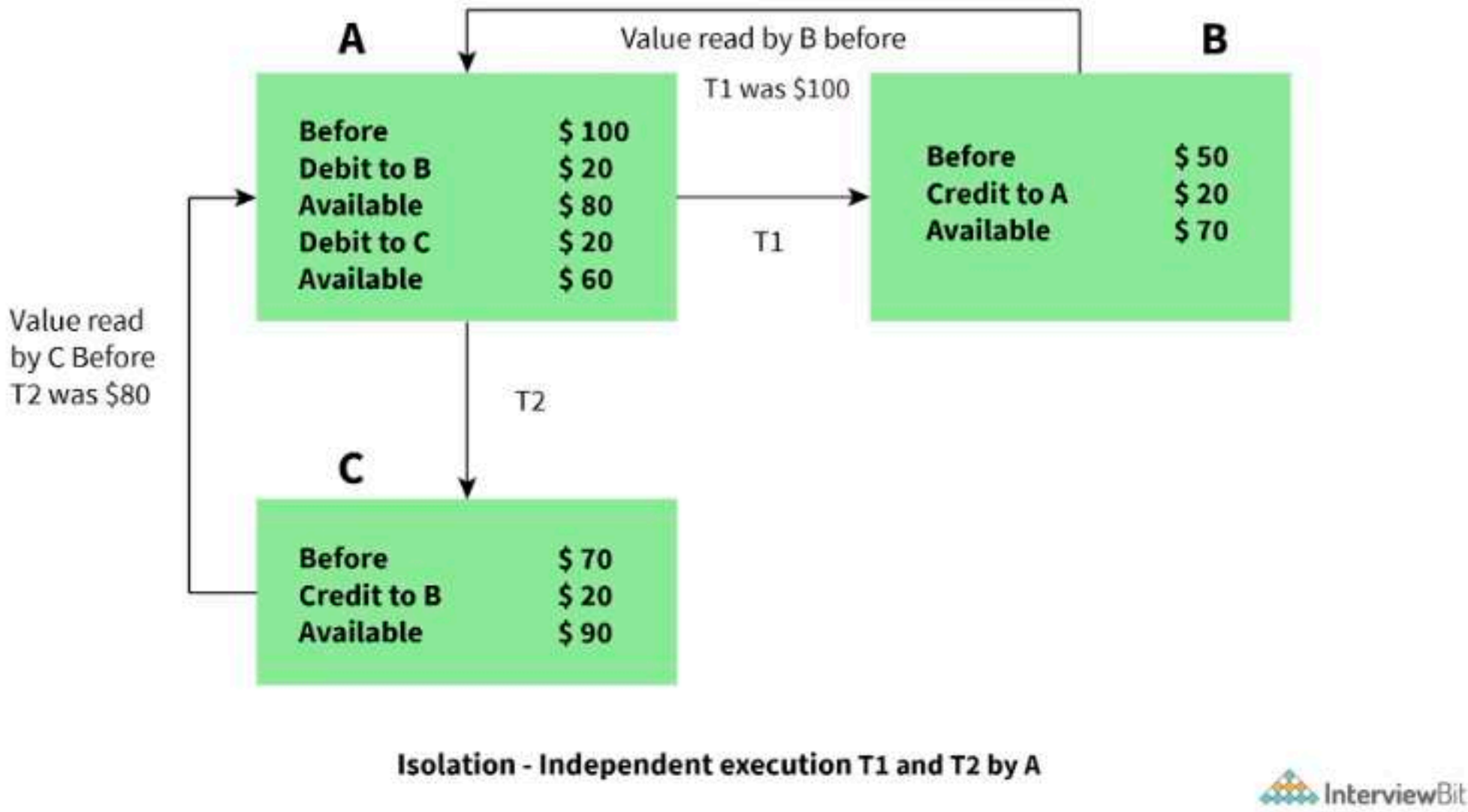
• **Consistency:** This property ensures that the data remains consistent before and after a transaction in a database.



- Consistency:** This property ensures that the data remains consistent before and after a transaction in a database.



- Isolation:** This property ensures that each transaction is occurring independently of the others. This implies that the state of an ongoing transaction doesn't affect the state of another ongoing transaction.



- Durability:** This property ensures that the data is not lost in cases of a system failure or restart and is present in the same state as it was before the system failure or restart.

7. Are NULL values in a database the same as that of blank space or zero?

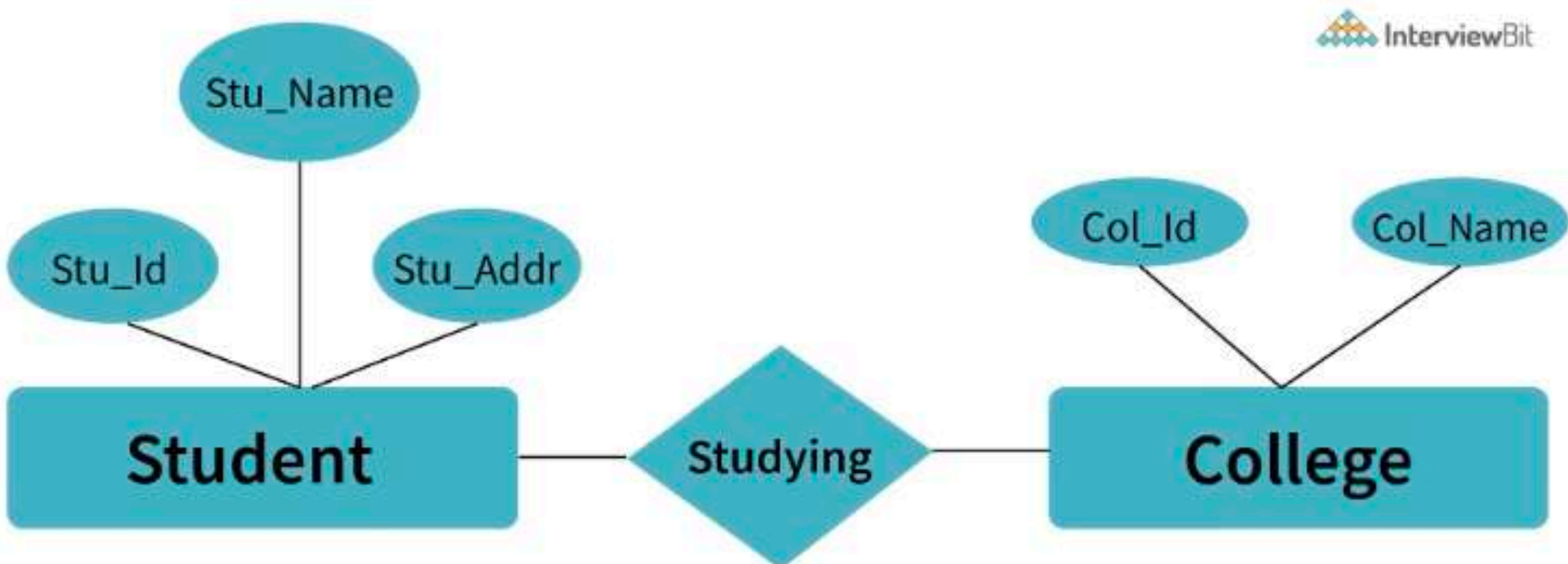
No, a NULL value is very different from that of zero and blank space as it represents a value that is assigned, unknown, unavailable, or not applicable as compared to blank space which represents a character and zero represents a number.

Example: NULL value in "number_of_courses" taken by a student represents that its value is unknown whereas 0 in it means that the student hasn't taken any courses.

Intermediate DBMS Interview Questions

8. What is meant by an entity-relationship (E-R) model? Explain the terms Entity, Entity Type, and Entity Set in DBMS.

An [entity-relationship model](#) is a diagrammatic approach to a database design where real-world objects are represented as entities and relationships between them are mentioned.

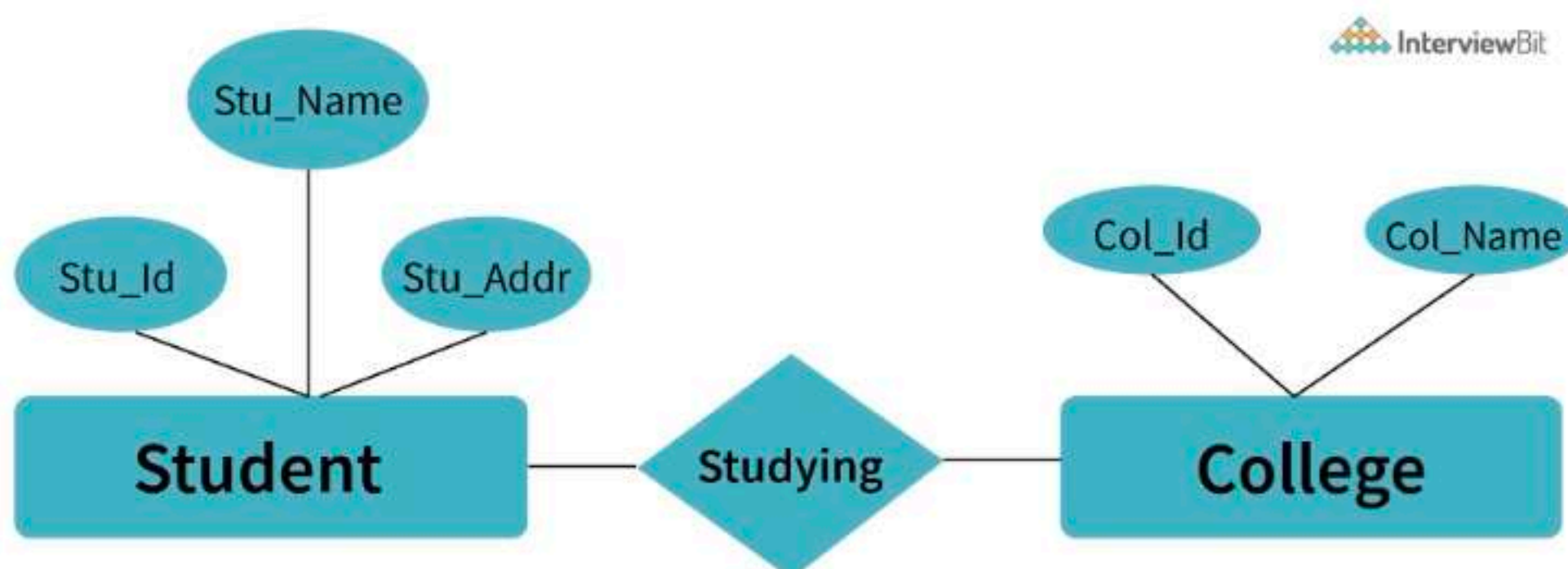


Example: NULL value in "number_of_courses" taken by a student represents that its value is unknown whereas 0 in it means that the student hasn't taken any courses.

Intermediate DBMS Interview Questions

8. What is meant by an entity-relationship (E-R) model? Explain the terms Entity, Entity Type, and Entity Set in DBMS.

An [entity-relationship model](#) is a diagrammatic approach to a database design where real-world objects are represented as entities and relationships between them are mentioned.



Sample E-R Diagram

- **Entity:** An entity is defined as a real-world object having attributes that represent characteristics of that particular object. For example, a student, an employee, or a teacher represents an entity.
- **Entity Type:** An entity type is defined as a collection of entities that have the same attributes. One or more related tables in a database represent an entity type. Entity type or attributes can be understood as a characteristic which uniquely identifies the entity. For example, a student represents an entity that has attributes such as student_id, student_name, etc.
- **Entity Set:** An entity set can be defined as a set of all the entities present in a specific entity type in a database. For example, a set of all the students, employees, teachers, etc. represent an entity set.

9. What is meant by normalization and denormalization?

[Normalization](#) is a process of reducing redundancy by organizing the data into multiple tables. Normalization leads to better usage of disk spaces and makes it easier to maintain the integrity of the database.

Denormalization is the reverse process of normalization as it combines the tables which have been normalized into a single table so that data retrieval becomes faster. JOIN operation allows us to create a denormalized form of the data by reversing the normalization.

10. What is a lock. Explain the major difference between a shared lock and an exclusive lock during a transaction in a database.

A database lock is a mechanism to protect a shared piece of data from getting updated by two or more database users at the same time. When a single database user or session has acquired a lock then no other database user or session can modify that data until the lock is released.

- **Shared Lock:** A shared lock is required for reading a data item and many transactions may hold a lock on the same data item in a shared lock. Multiple transactions are allowed to read the data items in a shared lock.
- **Exclusive lock:** An exclusive lock is a lock on any transaction that is about to perform a write operation. This type of lock doesn't allow more than one transaction and hence prevents any inconsistency in the database.

11. Explain the difference between the DELETE and TRUNCATE command in a DBMS.

DELETE command: this command is needed to delete rows from a table based on the condition provided by the WHERE clause.

- It deletes only the rows which are specified by the WHERE clause.
- It can be rolled back if required.
- It maintains a log to lock the row of the table before deleting it and hence it's slow.

TRUNCATE command: this command is needed to remove complete data from a table in a database. It is like a DELETE command which has no WHERE clause.

- It removes complete data from a table in a database.
- It can't be rolled back even if required. (truncate can be rolled back in some databases depending on their version but it can be tricky and can lead to data loss). Check this [link](#) for more details
- It doesn't maintain a log and deletes the whole table at once and hence it's fast.

12. Explain the difference between intension and extension in a database.

Following is the major difference between intension and extension in a database:

- **Intension:** Intension or popularly known as database schema is used to define the description of the database and is specified during the design of the database and mostly remains unchanged.
- **Extension:** Extension on the other hand is the measure of the number of tuples in a relation at a particular time. The extension of a database is also referred to as the snapshot of the database.

It can be rolled back even in regular queries (triggers can be rolled back in some databases depending on their version but it can be tricky and can lead to data loss). Check this [link](#) for more details

- It doesn't maintain a log and deletes the whole table at once and hence it's fast.

12. Explain the difference between intension and extension in a database.

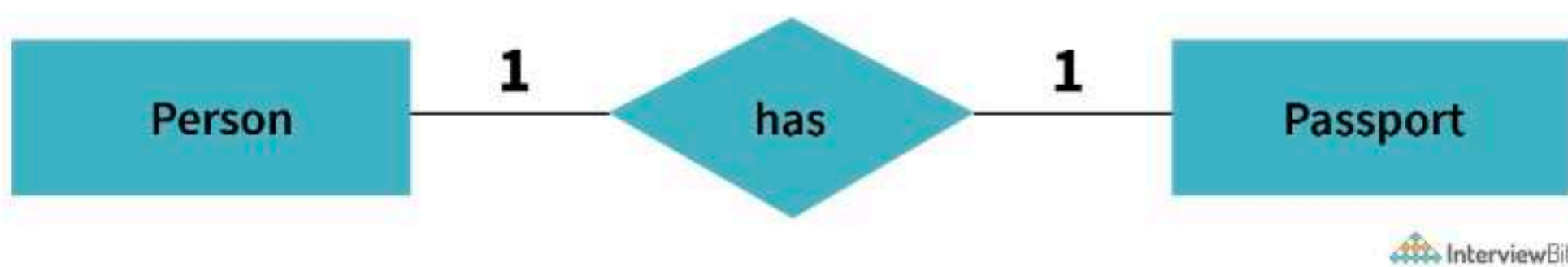
Following is the major difference between intension and extension in a database:

- **Intension:** Intension or popularly known as database schema is used to define the description of the database and is specified during the design of the database and mostly remains unchanged.
- **Extension:** Extension on the other hand is the measure of the number of tuples present in the database at any given point in time. The extension of a database is also referred to as the snapshot of the database and its value keeps changing as and when the tuples are created, updated, or destroyed in a database.

13. Explain different types of relationships amongst tables in a DBMS.

Following are different types of relationship amongst tables in a DBMS system:

- **One to One Relationship:** This type of relationship is applied when a particular row in table X is linked to a singular row in table Y.



InterviewBit

- **One to Many Relationship:** This type of relationship is applied when a single row in table X is related to many rows in table Y.



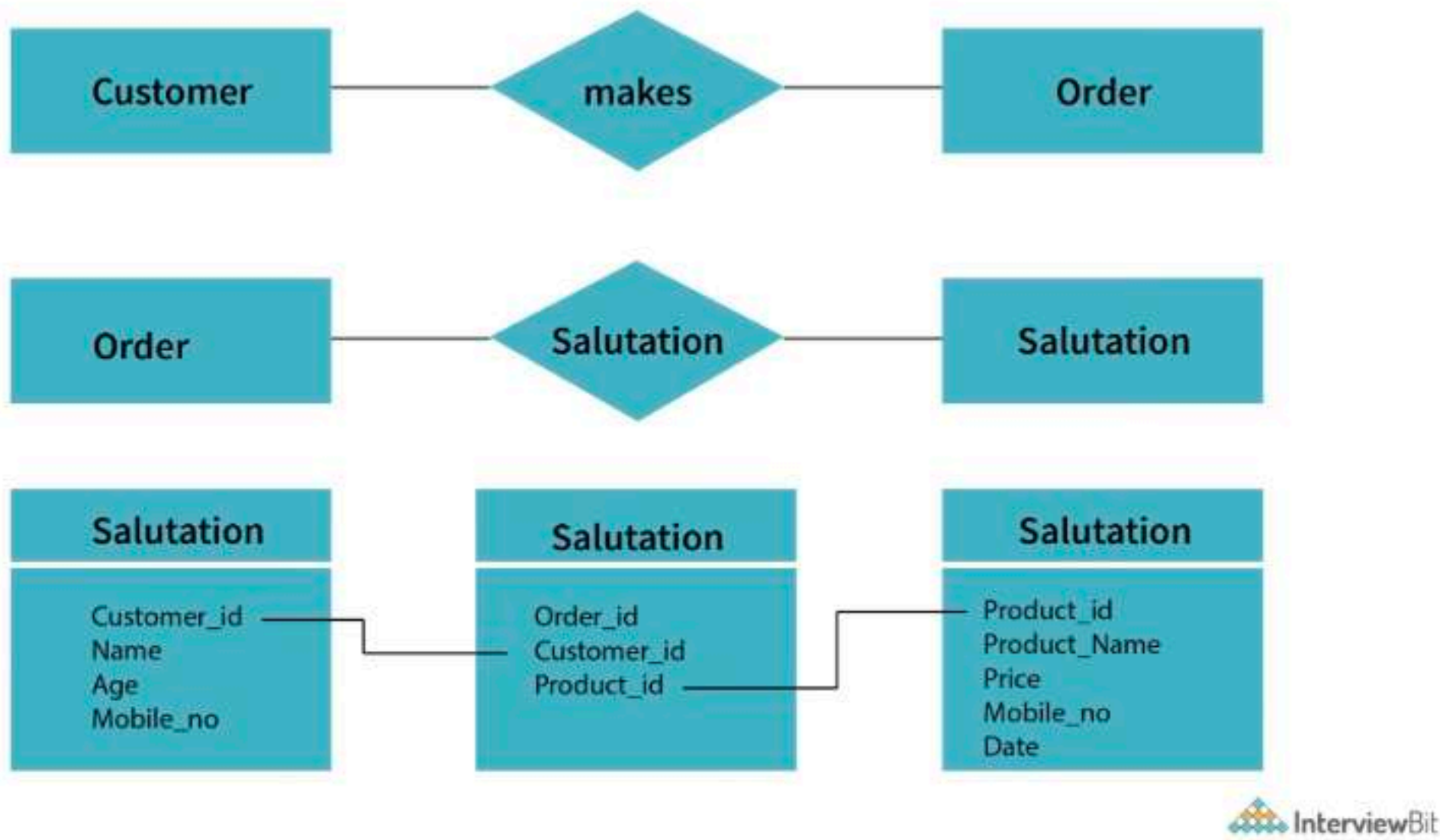
InterviewBit

- **Many to Many Relationship:** This type of relationship is applied when multiple rows in table X can be linked to multiple rows in table Y.



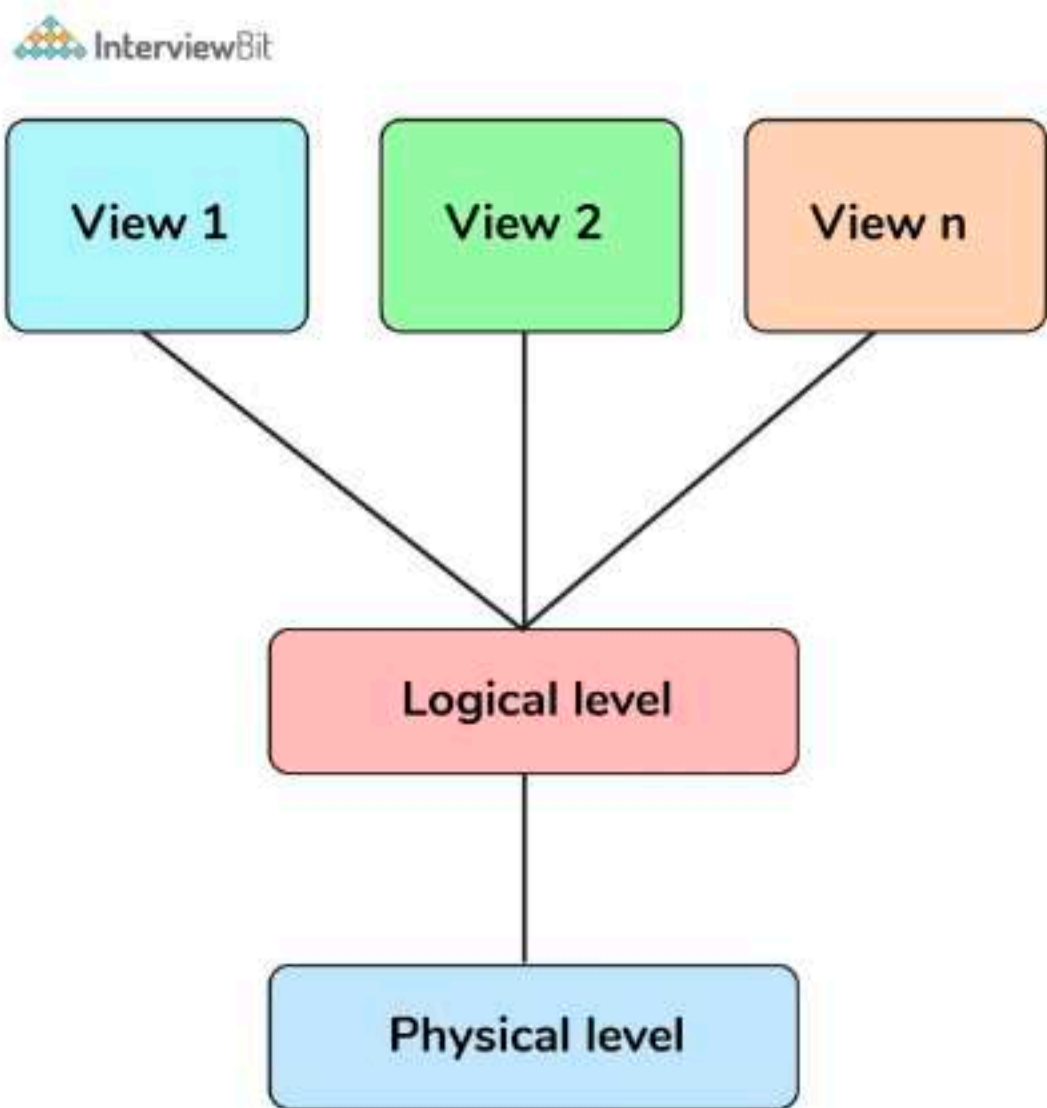
InterviewBit

- **Self Referencing Relationship:** This type of relationship is applied when a particular row in table X is associated with the same table.



14. Explain different levels of data abstraction in a DBMS.

The process of hiding irrelevant details from users is known as data abstraction. Data abstraction can be divided into 3 levels:

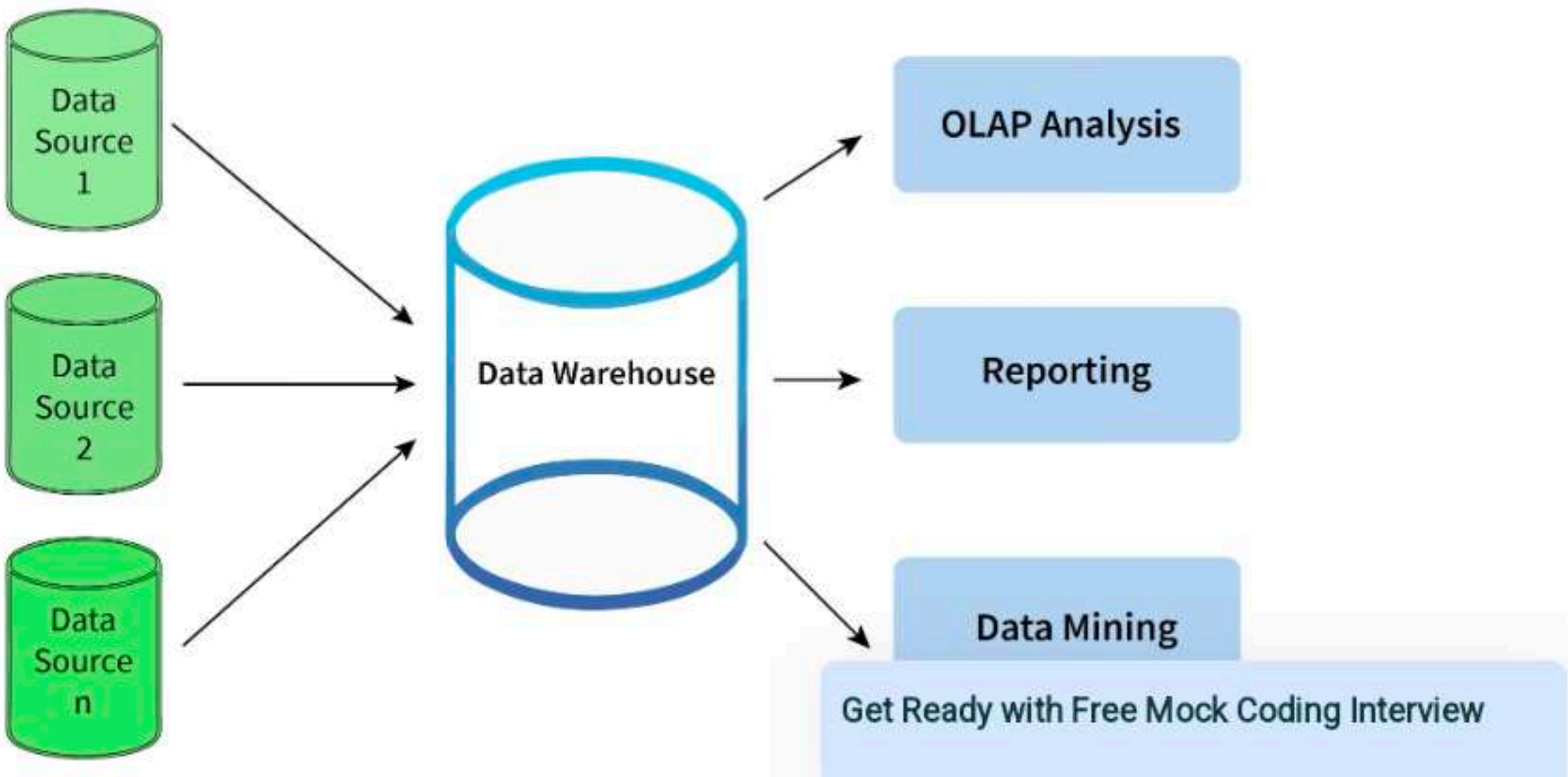


Three levels of data abstraction

- **Physical Level:** it is the lowest level and is managed by DBMS. This level consists of data storage descriptions and the details of this level are typically hidden from system admins, developers, and users.
- **Conceptual or Logical level:** it is the level on which developers and system admins work and it determines what data is stored in the database and what is the relationship between the data points.
- **External or View level:** it is the level that describes only part of the database and hides the details of the table schema and its physical storage from the users. The result of a query is an example of View level data abstraction. A view is a virtual table created by selecting fields from one or more tables present in the database.

15. What is Data Warehousing?

The process of collecting, extracting, transforming, and loading data from multiple sources and storing them in one database is known as [data warehousing](#). A data warehouse can be considered as a central repository where data flows from transactional systems and other relational databases and is used for data analytics. A data warehouse comprises a wide variety of an organization's historical data that supports the decision-making process in an organization.

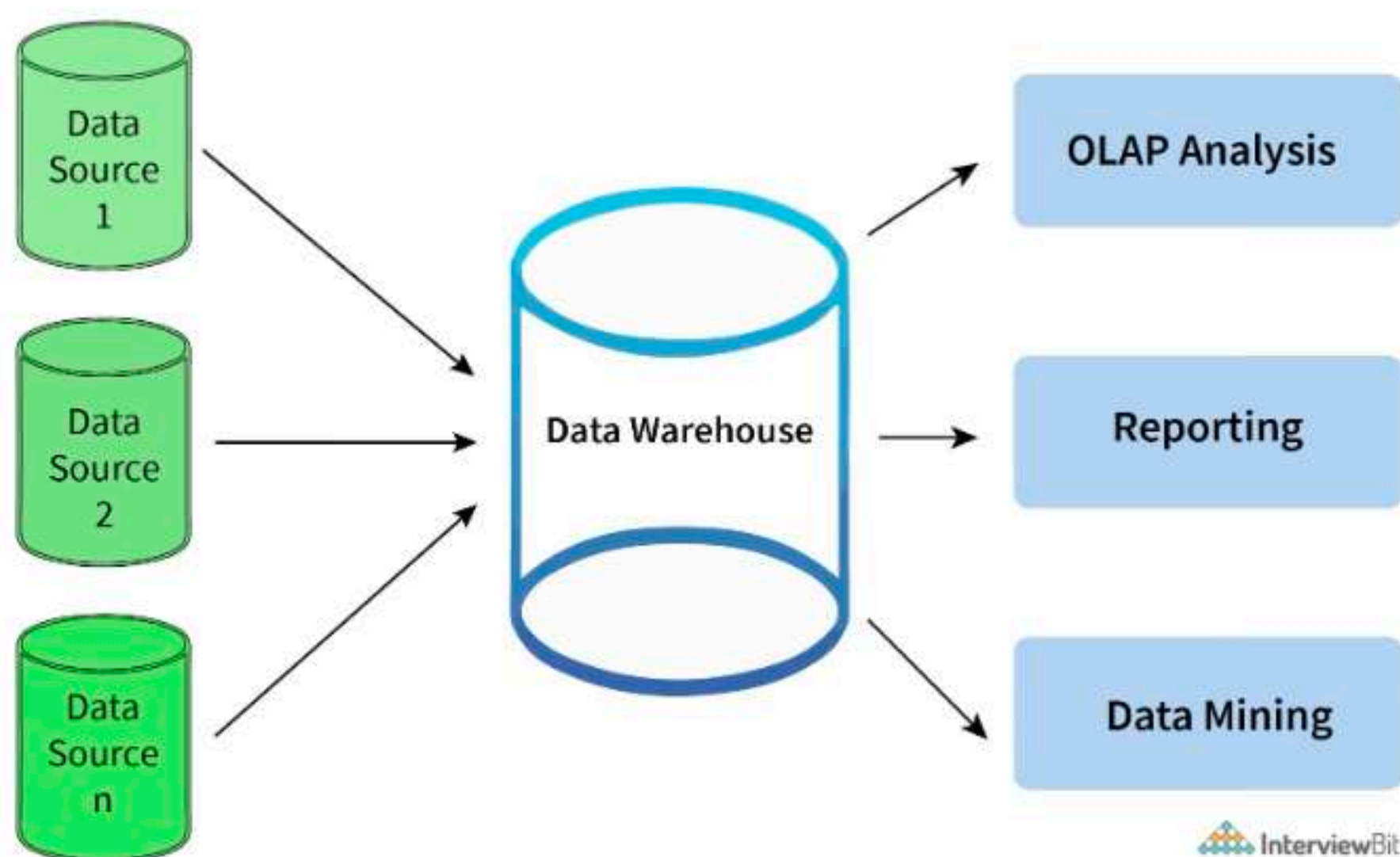


the database and what is the relationship between the data points.

- **External or View level:** it is the level that describes only part of the database and hides the details of the table schema and its physical storage from the users. The result of a query is an example of View level data abstraction. A view is a virtual table created by selecting fields from one or more tables present in the database.

15. What is Data Warehousing?

The process of collecting, extracting, transforming, and loading data from multiple sources and storing them in one database is known as [data warehousing](#). A data warehouse can be considered as a central repository where data flows from transactional systems and other relational databases and is used for data analytics. A data warehouse comprises a wide variety of an organization's historical data that supports the decision-making process in an organization.

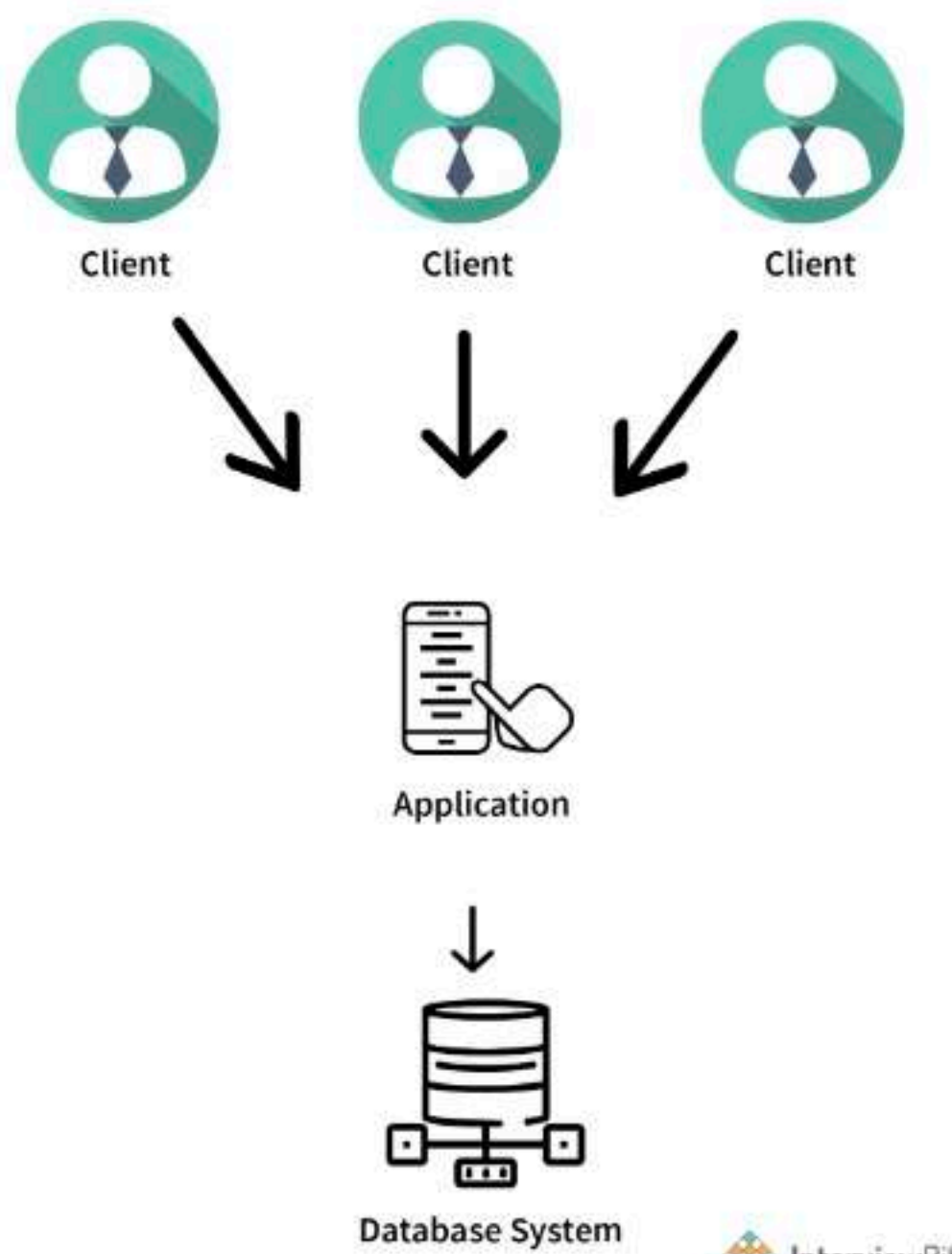


Advanced DBMS Interview Questions

16. Explain the difference between a 2-tier and 3-tier architecture in a DBMS.

The **2-tier architecture** refers to the client-server architecture in which applications at the client end directly communicate with the database at the server end without any middleware involved.

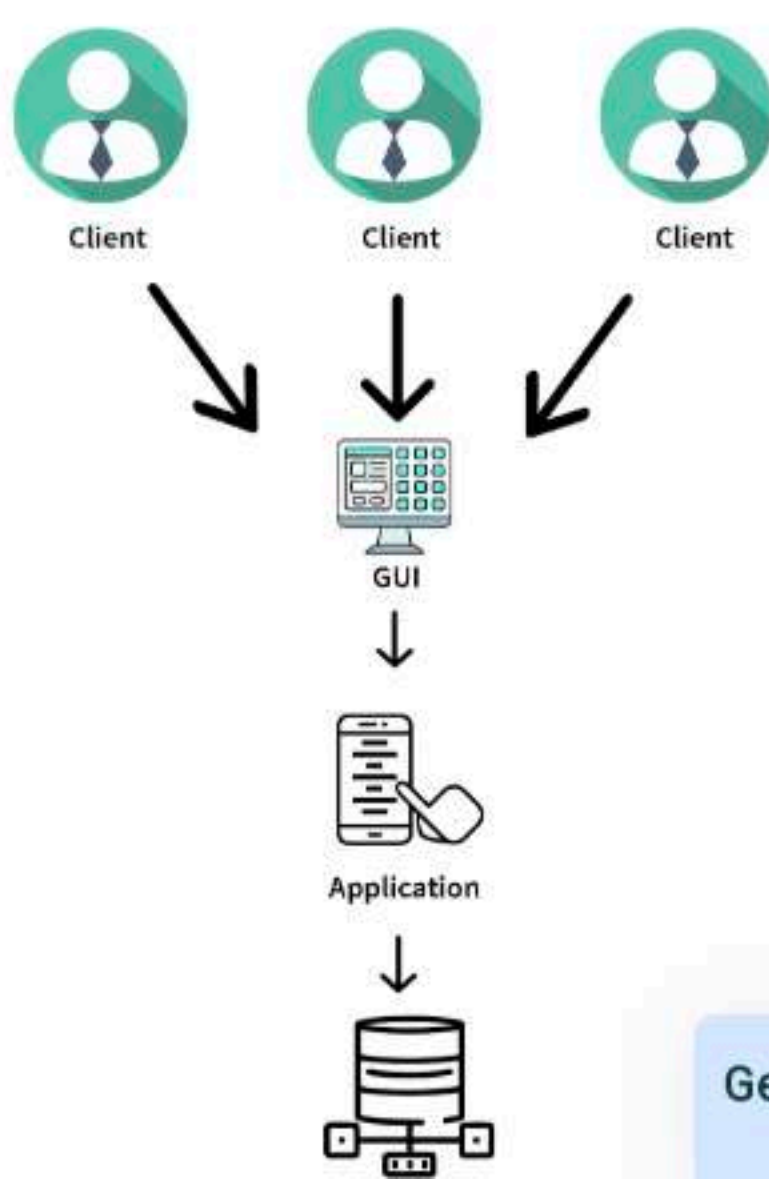
Example – Contact Management System created using MS-Access or Railway Reservation System, etc.



The above picture represents a 2-tier architecture in a DBMS.

The **3-tier architecture** contains another layer between the client and the server to provide GUI to the users and make the system much more secure and accessible. In this type of architecture, the application present on the client end interacts with an application on the server end which further communicates with the database system.

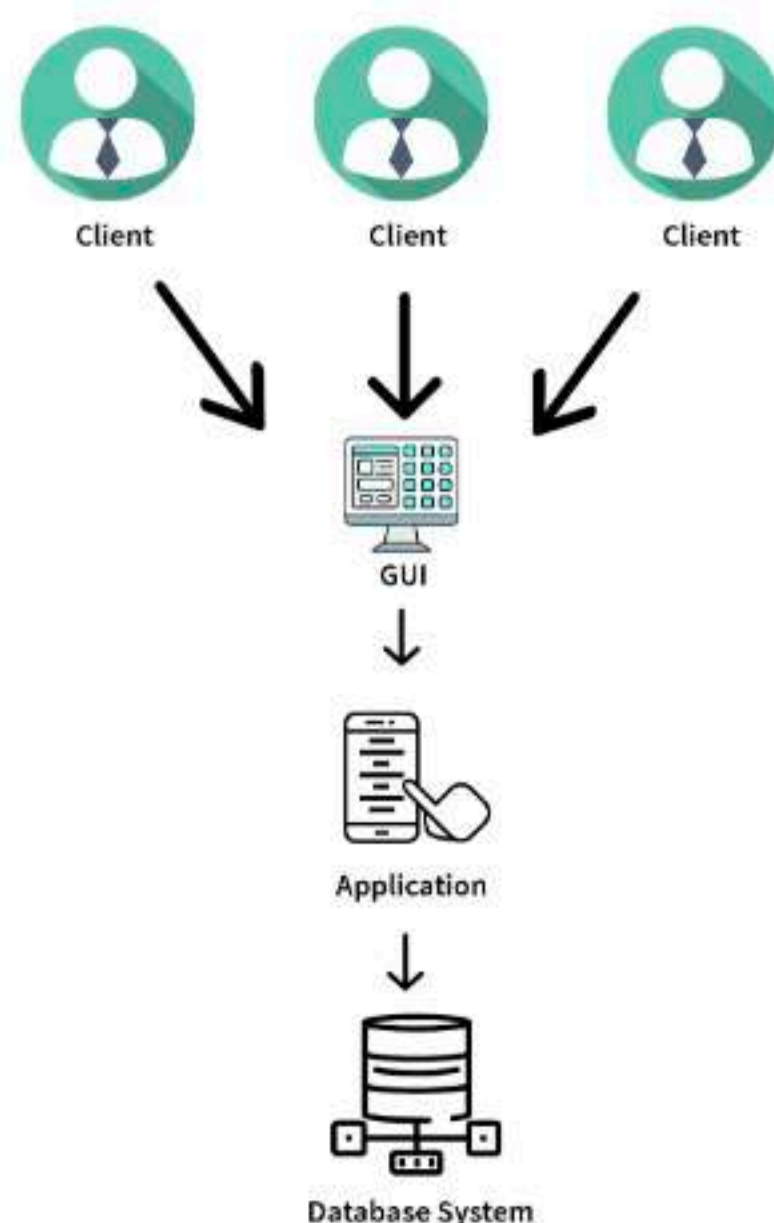
Example – Designing registration form which contains a text box, label, button or a large website on the Internet, etc.



The above picture represents a 2-tier architecture in a DBMS.

The **3-tier architecture** contains another layer between the client and the server to provide GUI to the users and make the system much more secure and accessible. In this type of architecture, the application present on the client end interacts with an application on the server end which further communicates with the database system.

Example – Designing registration form which contains a text box, label, button or a large website on the Internet, etc.



The above picture represents a 3-tier architecture in a DBMS.

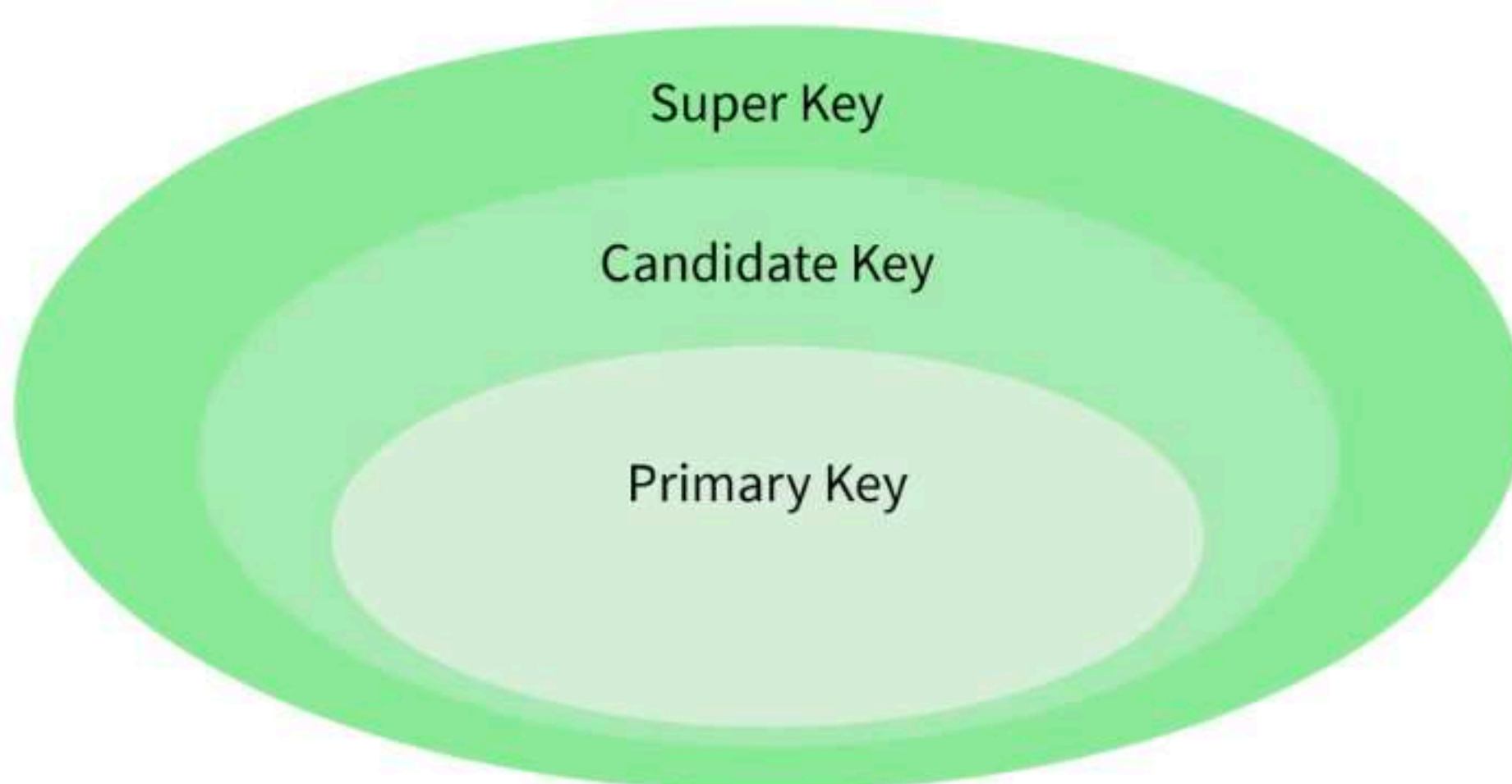
Recommended Tutorials:

- [DBMS MCQ with Answers](#)
- [DBMS Architecture](#)
- [Top Applications of DBMS](#)
- [SQL Interview Questions](#)
- [SQL Server Interview Questions](#)
- [SQL Queries Interview Questions](#)
- [MySQL Interview Questions](#)
- [MongoDB Interview Questions](#)
- [PL SQL Interview Questions](#)
- [Database Testing Interview Questions](#)
- [File System vs DBMS](#)
- [Interview Preparation Guide](#)

17. Explain different types of keys in a database.

There are mainly 7 types of keys in a database:

- **Candidate Key:** The candidate key represents a set of properties that can uniquely identify a table. Each table may have multiple candidate keys. One key amongst all candidate keys can be chosen as a primary key. In the below example since studentId and firstName can be considered as a Candidate Key since they can uniquely identify every tuple.
- **Super Key:** The super key defines a set of attributes that can uniquely identify a tuple. Candidate key and primary key are subsets of the super key, in other words, the super key is their superset.



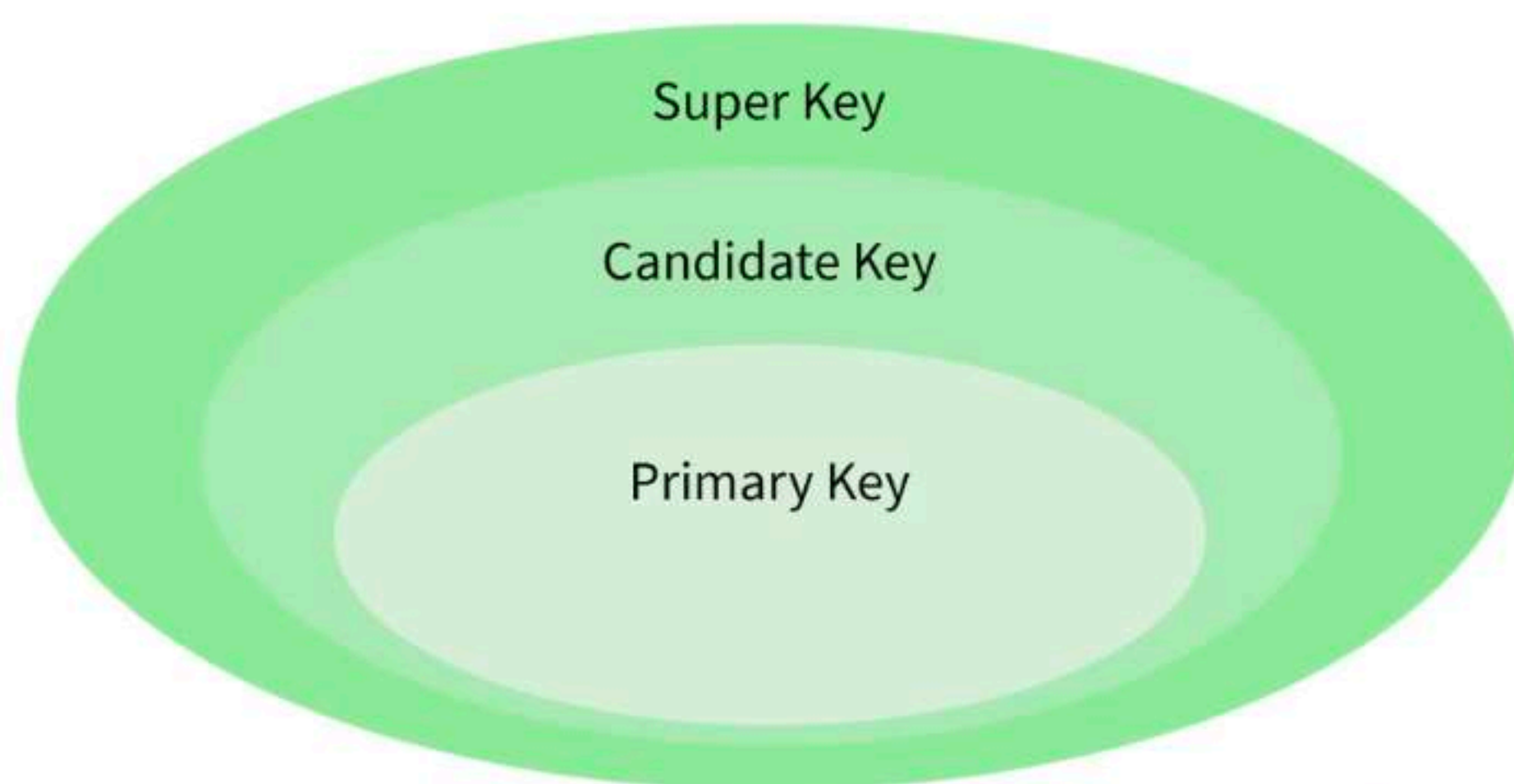
- **Primary Key:** The primary key defines a set of attributes that are used to uniquely identify every tuple. In the below example studentId and firstName are candidate keys and any one of them can be chosen as a Primary Key. In the given example studentId is chosen as the primary key for the student table.
- **Unique Key:** The unique key is very similar to the primary key except that primary keys don't allow NULL values in the column but unique keys allow them. So essentially unique keys are primary keys with NULL values.
- **Alternate Key:** All the candidate keys which are not chosen as primary keys are considered as alternate Keys. In the below example, firstName and lastName are alternate keys in the database.
- **Foreign Key:** The foreign key defines an attribute that can only take the values present in another table. In the below example courseId from the Student table

- [Database Testing Interview Questions](#)
- [File System vs DBMS](#)
- [Interview Preparation Guide](#)

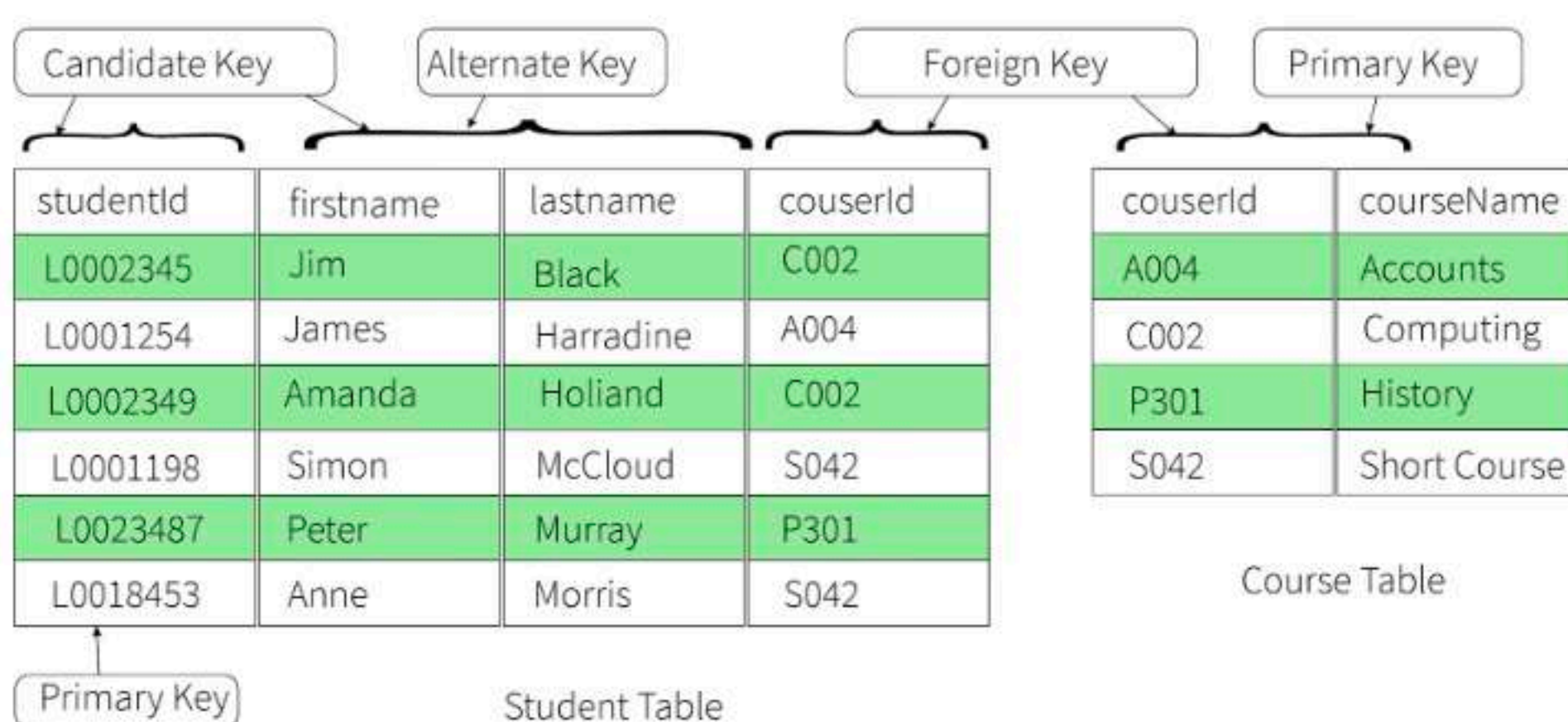
17. Explain different types of keys in a database.

There are mainly 7 types of keys in a database:

- **Candidate Key:** The candidate key represents a set of properties that can uniquely identify a table. Each table may have multiple candidate keys. One key amongst all candidate keys can be chosen as a primary key. In the below example since studentId and firstName can be considered as a Candidate Key since they can uniquely identify every tuple.
- **Super Key:** The super key defines a set of attributes that can uniquely identify a tuple. Candidate key and primary key are subsets of the super key, in other words, the super key is their superset.



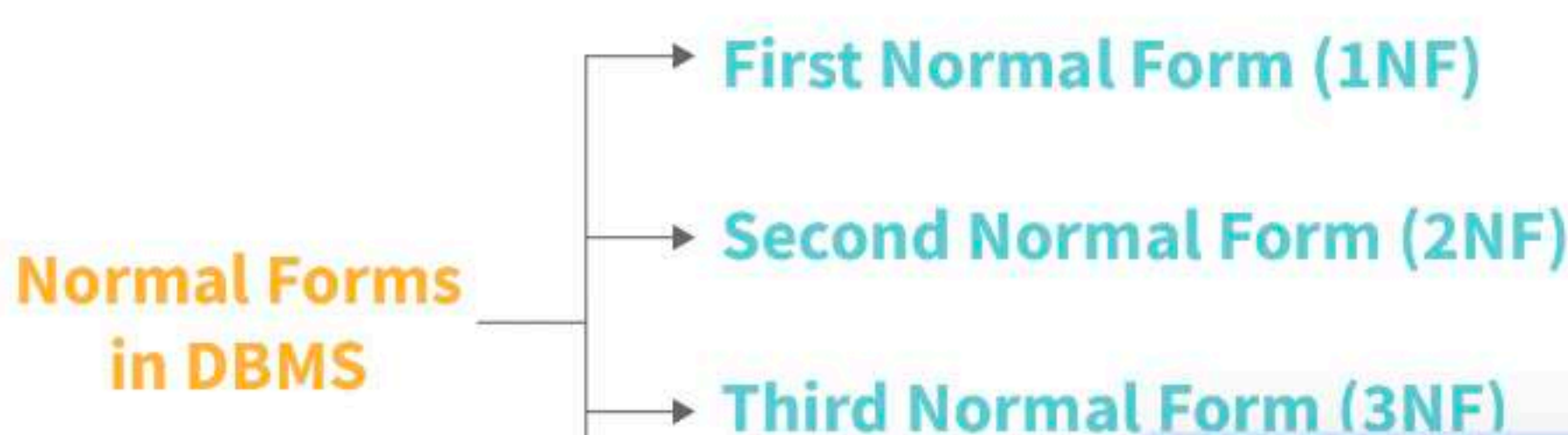
- **Primary Key:** The primary key defines a set of attributes that are used to uniquely identify every tuple. In the below example studentId and firstName are candidate keys and any one of them can be chosen as a Primary Key. In the given example studentId is chosen as the primary key for the student table.
- **Unique Key:** The unique key is very similar to the primary key except that primary keys don't allow NULL values in the column but unique keys allow them. So essentially unique keys are primary keys with NULL values.
- **Alternate Key:** All the candidate keys which are not chosen as primary keys are considered as alternate Keys. In the below example, firstName and lastName are alternate keys in the database.
- **Foreign Key:** The foreign key defines an attribute that can only take the values present in one table common to the attribute present in another table. In the below example couserId from the Student table is a foreign key to the Course table, as both, the tables contain couserId as one of their attributes.
- **Composite Key:** A composite key refers to a combination of two or more columns that can uniquely identify each tuple in a table. In the below example the studentId and firstName can be grouped to uniquely identify every tuple in the table.



Relationship Between Keys

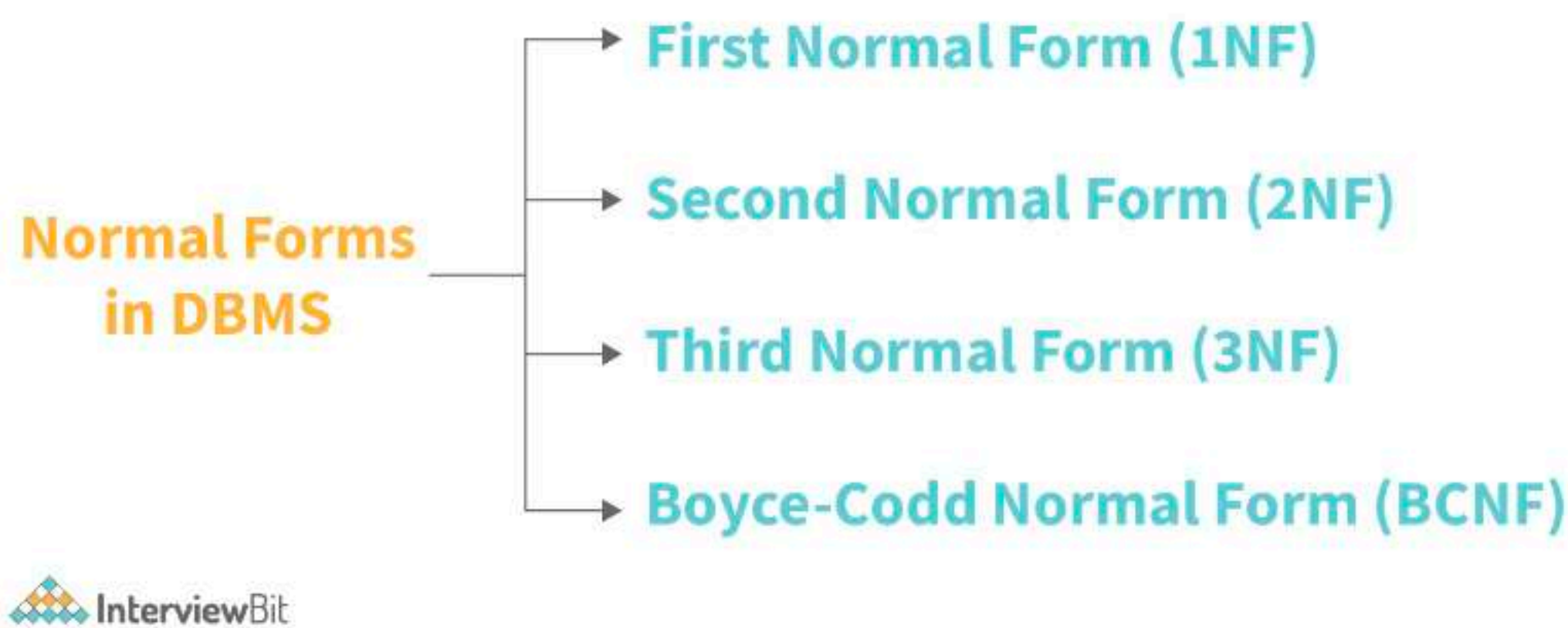
18. Explain different types of Normalization forms in a DBMS.

Following are the major normalization forms in a DBMS:



18. Explain different types of Normalization forms in a DBMS.

Following are the major normalization forms in a DBMS:



Full Names	Physical Address	Movies Rented	Salutation
Janet Jones	First street Plot No 4	Pirates. the Caribbean Clash of the Titans	Ms.
Robert Phil	3rd street 34	Forgetting Sarah Marshal Daddy's Little Girls	Mr.
Robert Phil	5th Avenue	Clash of the Titans	Mr.

Considering the above Table-1 as the reference example for understanding different normalization forms.

- 1NF: It is known as the first normal form and is the simplest type of normalization that you can implement in a database. A table to be in its first normal form should satisfy the following conditions:
 - Every column must have a single value and should be atomic.
 - Duplicate columns from the same table should be removed.
 - Separate tables should be created for each group of related data and each row should be identified with a unique column.

Full Names	Physical Address	Movies Rented	Salutation
Janet Jones	First street Plot No 4	Pirates. the Caribbean	Ms.
Janet Jones	First street Plot No 4	Clash of the Titans	Ms.
Robert Phil	3rd street 34	Forgetting Sarah Marshal	Mr.
Robert Phil	3rd street 34	Daddy's Little Girls	Mr.
Robert Phil	5th Avenue	Clash of the Titans	Mr.

Table-1 converted to 1NF form

- 2NF: It is known as the second normal form. A table to be in its second normal form should satisfy the following conditions:
 - The table should be in its 1NF i.e. satisfy all the conditions of 1NF.
 - Every non-prime attribute of the table should be fully functionally dependent on the primary key i.e. every non-key attribute should be dependent on the primary key in such a way that if any key element is deleted then even the non_key element will be saved in the database.

Table-1 converted to 1NF form

- 2NF: It is known as the second normal form. A table to be in its second normal form should satisfy the following conditions:
 - The table should be in its 1NF i.e. satisfy all the conditions of 1NF.
 - Every non-prime attribute of the table should be fully functionally dependent on the primary key i.e. every non-key attribute should be dependent on the primary key in such a way that if any key element is deleted then even the non_key element will be saved in the database.

Membership ID	Full Names	Physical Address	Salutation
1	Janet Jones	First street Plot No 4	Ms.
2	Robert Phil	3rd street 34	Mr.
3	Robert Phil	5th Avenue	Mr.



Membership ID	Movies Rented
1	Pirates. the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans



Breaking Table-1 into 2 different tables to move it to 2NF.

- 3NF: It is known as the third normal form. A table to be in its third normal form should satisfy the following conditions:
 - The table should be in its 2NF i.e. satisfy all the conditions of 2NF.
 - There is no transitive functional dependency of one attribute on any attribute in the same table.

Membership ID	Full Names	Physical Address	SalutationID
1	Janet Jones	First street Plot No 4	2
2	Robert Phil	3rd street 34	1
3	Robert Phil	5th Avenue	1



Membership ID	Movies Rented
1	Pirates. the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

Membership ID	Movies Rented
1	Pirates. the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans



Salutation ID	Salutation
1	Mr.
2	Ms.
3	Mrs.
4	Dr.



Breaking Table-1 into 3 different tables to move it to 3NF.

- **BCNF:** BCNF stands for [Boyce-Codd Normal Form](#) and is an advanced form of 3NF. It is also referred to as 3.5NF for the same reason. A table to be in its BCNF normal form should satisfy the following conditions:
 - The table should be in its 3NF i.e. satisfy all the conditions of 3NF.
 - For every functional dependency of any attribute A on B (A->B), A should be the super key of the table. It simply implies that A can't be a non-prime attribute if B is a prime attribute.

DBMS MCQ

1.A record in a relational DBMS is also known as:

☐ Attribute

☐ Tuple

☐ Relation

☐ Criteria

2.What does ODBC in a database stand for:

☐ Oral Database Connectivity

☐ Oracle Database Connectivity

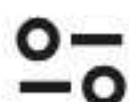
☐ Open Database Connectivity

☐ Object Database Connectivity

3. In a DBMS, TCL means:

☐ Ternary Control Language

☐



Top 60 DBMS Interview Questions with Answers for 2025

Last Updated : 28 Apr, 2025



A **Database Management System (DBMS)** is the backbone of modern data storage and management. Understanding DBMS concepts is critical for anyone looking to work with databases. Whether you're preparing for your first job in database management or advancing in your career, being well-prepared for a DBMS interview can make all the difference.

In this article, we've covered a list of **DBMS interview questions** that cover everything from basic to advanced topics. These questions will help us build a solid understanding of DBMS concepts, from how databases are structured to complex query optimization. Start your preparation today to secure your dream role in database management!

Basic DBMS Interview Questions

DBMS Basic Interview Questions are designed to test your foundational understanding of database management systems. These questions focus on key concepts such as the differences between relational and non-relational databases, basic DBMS operations, data types, and database structure. A strong grasp of these basics will help you confidently approach more advanced topics.

1. What is a Database Management System (DBMS)?

A **Database Management System (DBMS)** is software that is used to manage and organize databases. It provides an interface to interact with the data stored in a database. The DBMS is responsible for tasks such as storing, retrieving, and updating data, ensuring data integrity, security, and managing concurrency. Examples include MySQL, PostgreSQL, Oracle, and SQL Server.

2. What are the advantages of using a DBMS?

The advantages of using a DBMS are:

- **Data Integrity:** Ensures that the data is accurate and consistent.
- **Data Security:** Provides controlled access to sensitive data by setting permissions for different users.
- **Efficient Data Retrieval:** Optimizes queries and indexing, allowing faster data retrieval.
- **Reduced Redundancy:** Avoids duplicate data by enforcing normalization.
- **Backup and Recovery:** Offers automatic backup and recovery mechanisms.
- **Concurrent Access:** Allows multiple users to access the database at the same time without conflicts.

3. What is the difference between DBMS and RDBMS?

- **DBMS (Database Management System):** A system that allows users to create, store, modify, and delete data. It does not require a relational structure for data organization. Examples: Microsoft Access, XML databases.
- **RDBMS (Relational Database Management System):** A subset of DBMS that stores data in a structured format, using tables (relations), and supports relational operations like joins. It enforces data integrity through keys and supports SQL for querying. Examples: MySQL, Oracle, SQL Server.

4. What are the different types of DBMS?

The four types of DBMS are:

- **Hierarchical DBMS:** Data is organized in a tree-like structure with parent-child relationships. Example: IBM's IMS.
- **Network DBMS:** Data is represented as a graph with many-to-many relationships. Example: Integrated Data Store (IDS).
- **Relational DBMS (RDBMS):** Data is organized in tables (relations) and managed through SQL. Example: MySQL, PostgreSQL.
- **Object-Oriented DBMS:** Data is stored as objects, like in object-oriented programming. Example: ObjectDB.

5. What is a relation in DBMS?

A **relation** in DBMS is a table that consists of rows and columns. Each row represents a record, and each column represents an attribute or property of the entity being described. Relations are defined by a schema, which specifies the attributes of the table.

data but presents data from other tables.

16. What are the different types of relationships in DBMS?

The three main types of relationships in DBMS are:

- **One-to-One (1:1):** A record in one table is associated with a single record in another table.
- **One-to-Many (1:M):** A record in one table is associated with multiple records in another table.
- **Many-to-Many (M:M):** Multiple records in one table are associated with multiple records in another table.

17. Explain the concept of a schema in DBMS.

A **schema** in DBMS is the structure that defines the organization of data in a database. It includes tables, views, relationships, and other elements. A schema defines the tables and their columns, along with the constraints, keys, and relationships.

18. What are constraints in DBMS?

Constraints in DBMS are rules that limit the type of data that can be inserted into a table to ensure data integrity and consistency. Common types of constraints include: **NOT NULL, PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT.**

19. What is the difference between DELETE and TRUNCATE in SQL?

- **DELETE:** Deletes specific rows from a table based on a condition. It logs each row deletion and can be rolled back.
- **TRUNCATE:** Removes all rows from a table without logging individual row deletions. It cannot be rolled back and is faster than DELETE.

20. What is an index in DBMS and how is it used?

An **index** is a data structure that improves the speed of data retrieval operations on a database table. It works like a table of contents in a book, allowing the database to quickly find the location of a record based on a column value.

21. What is the role of the Database Administrator (DBA)?

A **Database Administrator (DBA)** is responsible for managing and overseeing the entire database environment. Their key responsibilities include:

- **Database Design:** Structuring the database for optimal storage and performance.
- **Backup and Recovery:** Ensuring regular backups and providing recovery solutions in case of failures.
- **Performance Tuning:** Monitoring and optimizing the database's performance.
- **Security Management:** Managing user access, privileges, and enforcing security policies.
- **Data Integrity:** Ensuring data consistency and integrity through constraints and checks.
- **Upgrades and Patches:** Keeping the database software up-to-date with patches and upgrades.
- **Troubleshooting:** Identifying and resolving database-related issues.

22. What is an entity-relationship diagram (ERD)?

An **Entity-Relationship Diagram (ERD)** is a visual representation of the entities within a system and the relationships between those entities. It is used in database design to model the structure of data and how different pieces of data relate to each other.

- **Entities:** Objects or things within the system (e.g., Student, Course).
- **Attributes:** Properties or details about an entity (e.g., Student Name, Course Duration).
- **Relationships:** How entities interact with each other (e.g., Student enrolls in Course).

Example of ERD:

- A Student entity might have attributes like ID, Name, and Age.
- A Course entity might have attributes like CourseID, CourseName.
- A relationship Enrolls connects Student and Course with attributes like EnrollmentDate.

23. What is a join in SQL? Name and explain different types of joins.

A **JOIN** in SQL is an operation that combines columns from two or more tables based on a related column between them. Joins are used to query data from multiple tables in a relational database.

Here are the different types of **joins** in SQL:

1. **INNER JOIN:** Returns only the rows where there is a match in both tables.

Example:

- A relationship Enrolls connects Student and Course with attributes like EnrollmentDate.

23. What is a join in SQL? Name and explain different types of joins.

A **JOIN** in SQL is an operation that combines columns from two or more tables based on a related column between them. Joins are used to query data from multiple tables in a relational database.

Here are the different types of **joins** in SQL:

1. **INNER JOIN:** Returns only the rows where there is a match in both tables.

Example:

```
SELECT * FROM Student
INNER JOIN Course ON Student.ID = Course.StudentID;
```

2. **LEFT JOIN (or LEFT OUTER JOIN):** Returns all rows from the left table and the matching rows from the right table. If there is no match, NULL values will be returned for columns from the right table.

Example:

```
SELECT * FROM Student
LEFT JOIN Course ON Student.ID = Course.StudentID;
```

3. **RIGHT JOIN (or RIGHT OUTER JOIN):** Returns all rows from the right table and the matching rows from the left table. If there is no match, NULL values will be returned for columns from the left table.

Example:

```
SELECT * FROM Student
RIGHT JOIN Course ON Student.ID = Course.StudentID;
```

4. **FULL JOIN (or FULL OUTER JOIN):** Returns all rows when there is a match in either the left or the right table. If there is no match, NULL values will be returned for the columns of the table without a match.

Example:

```
SELECT * FROM Student
FULL JOIN Course ON Student.ID = Course.StudentID;
```

5. **CROSS JOIN:** Returns the Cartesian product of two tables, i.e., every combination of rows from both tables.

Example:

```
SELECT * FROM Student
CROSS JOIN Course;
```

6. **SELF JOIN:** A **SELF JOIN** is a join where a table is joined with itself. It is used when we need to compare rows within the same table. To differentiate the two instances of the same table, aliases are used.

Example:

```
SELECT E1.Employee_ID, E1.Employee_Name, E2.Employee_Name AS Manager_Name
FROM Employee E1
LEFT JOIN Employee E2 ON E1.Manager_ID = E2.Employee_ID;
```

24. What is a subquery in SQL? Provide an example.

A **subquery** in SQL is a query embedded within another query. It is used to retrieve data that will be used in the outer query. Subqueries can be used in SELECT, INSERT, UPDATE, or DELETE statements.

There are two types of subqueries:

- **Single-row subquery:** Returns a single value.
- **Multiple-row subquery:** Returns multiple values.

Example of a subquery: To find the names of students who have a higher age than the average age:

```
SELECT Name FROM Student
WHERE Age > (SELECT AVG(Age) FROM Student);
```



```
LEFT JOIN Employee E2 ON E1.Manager_ID = E2.Employee_ID;
```

24. What is a subquery in SQL? Provide an example.

A **subquery** in SQL is a query embedded within another query. It is used to retrieve data that will be used in the outer query. Subqueries can be used in SELECT, INSERT, UPDATE, or DELETE statements.

There are two types of subqueries:

- **Single-row subquery:** Returns a single value.
- **Multiple-row subquery:** Returns multiple values.

Example of a subquery: To find the names of students who have a higher age than the average age:

```
SELECT Name FROM Student  
WHERE Age > (SELECT AVG(Age) FROM Student);
```

25. What are aggregate functions in SQL? Name a few examples.

Aggregate functions in SQL are functions that operate on a set of values (or a group of rows) and return a single result. They are often used in conjunction with the **GROUP BY** clause. Here are a few commonly used **aggregate functions** in SQL:

1. **COUNT():** Returns the number of rows or non-NULL values in a column

Example:

```
SELECT COUNT(*) FROM Student;
```

2. **SUM():** Returns the sum of values in a numeric column.

Example:

```
SELECT SUM(Amount) FROM Orders;
```

3. **AVG():** Returns the average value of a numeric column.

Example:

```
SELECT AVG(Salary) FROM Employees;
```

4. **MAX():** Returns the maximum value in a column

Example:

```
SELECT MAX(Age) FROM Student;
```

5. **MIN():** Returns the minimum value in a column.

Example:

```
SELECT MIN(Salary) FROM Employees;
```

Intermediate DBMS Interview Questions

DBMS Intermediate Interview Questions dive deeper into more complex DBMS concepts and practical applications. These questions assess your ability to handle real-world database problems involving transactions, concurrency control, and schema design. Topics such as joins, indexing, normalization, and transaction management are often discussed at this level.

26. What is a transaction in DBMS? What are the properties of a transaction?

A **transaction** in DBMS is a sequence of one or more SQL operations executed as a single unit of work. A transaction ensures data integrity, consistency, and isolation, and it guarantees that the database reaches a valid state, regardless of errors or system failures.

Properties of a transaction (ACID Properties):

- **Atomicity:** All operations within the transaction are completed successfully, or none are applied (i.e., the transaction is atomic).
- **Consistency:** The transaction brings the database from one valid state to another valid state.
- **Isolation:** The operations of one transaction are isolated from others; intermediate results are not visible to other transactions.
- **Durability:** Once a transaction is committed, its effects are permanent, even in the event of a system crash.


```
SELECT MIN(Salary) FROM Employees;
```

Intermediate DBMS Interview Questions

DBMS Intermediate Interview Questions dive deeper into more complex DBMS concepts and practical applications. These questions assess your ability to handle real-world database problems involving transactions, concurrency control, and schema design. Topics such as joins, indexing, normalization, and transaction management are often discussed at this level.

26. What is a transaction in DBMS? What are the properties of a transaction?

A **transaction** in DBMS is a sequence of one or more SQL operations executed as a single unit of work. A transaction ensures data integrity, consistency, and isolation, and it guarantees that the database reaches a valid state, regardless of errors or system failures.

Properties of a transaction (ACID Properties):

- **Atomicity:** All operations within the transaction are completed successfully, or none are applied (i.e., the transaction is atomic).
- **Consistency:** The transaction brings the database from one valid state to another valid state.
- **Isolation:** The operations of one transaction are isolated from others; intermediate results are not visible to other transactions.
- **Durability:** Once a transaction is committed, its effects are permanent, even in the event of a system crash.

27. What is the ACID property in DBMS?

ACID stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability**, which are the key properties that guarantee reliable transaction processing:

- **Atomicity:** All operations in a transaction are treated as a single unit. If one operation fails, the entire transaction fails and the database state remains unchanged.
- **Consistency:** Ensures the database starts and ends in a consistent state, with all rules and constraints enforced.
- **Isolation:** Transactions are executed independently, and the intermediate states of a transaction are invisible to other transactions.
- **Durability:** Once a transaction is committed, its changes are permanent, even if a system failure occurs.

28. Explain the concept of a stored procedure in DBMS.

A **stored procedure** is a precompiled collection of one or more SQL statements stored in the database. Stored procedures allow users to execute a series of operations as a single unit, improving performance and reusability. They can accept input parameters, perform operations, and return results.

Example:

```
CREATE PROCEDURE GetStudentDetails(IN student_id INT)
BEGIN
    SELECT * FROM Student WHERE ID = student_id;
END;
```

29. What are triggers in DBMS? Provide an example.

A **trigger** is a special kind of stored procedure that automatically executes (or "fires") in response to certain events on a table, such as insertions, updates, or deletions. Triggers are used to enforce business rules, maintain consistency, or log changes.

Example:

```
CREATE TRIGGER after_student_insert
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (Action, StudentID, ActionTime)
    VALUES ('INSERT', NEW.ID, NOW());
END;
```

30. What is the difference between UNION and UNION ALL in SQL?

- **UNION:** Combines the result of two queries and removes duplicate rows.
- **UNION ALL:** Combines the result of two queries but does **not** remove duplicates, thus it is faster than UNION.

determinant is a candidate key.

33. What is the difference between INNER JOIN and OUTER JOIN?

Aspect	INNER JOIN	OUTER JOIN
Result Set	Returns only matching rows from both tables.	Returns all rows from one or both tables, with NULL where no match is found.
Types	Single type.	Three types: LEFT, RIGHT, FULL.
Use Case	When you need only the intersecting data.	When you need to preserve all data, even with mismatches.
Performance	Generally faster as it deals with fewer rows.	Can be slower due to handling more rows and NULL values.
Reliability	Reliable for matching data across tables.	Reliable when you need to retain all data, but can introduce NULL-related issues.

34. What is data redundancy in a database? How can it be reduced?

Data redundancy refers to the unnecessary repetition of data in a database. It can lead to inconsistencies, increased storage requirements, and maintenance challenges.

Reduction methods:

- **Normalization:** Splitting large tables into smaller ones to eliminate redundancy.
- **Eliminating duplicate data:** Using constraints like UNIQUE and PRIMARY KEY to enforce data consistency.

35. What is a deadlock in DBMS? How can it be prevented?

A **deadlock** occurs when two or more transactions are blocked because each transaction is waiting for the other to release resources. This results in a situation where none of the transactions can proceed.

Prevention techniques:

- **Lock ordering:** Ensuring that all transactions acquire locks in the same predefined order.
- **Timeouts:** Automatically rolling back transactions that have been waiting too long for resources.
- **Deadlock detection:** Periodically checking for deadlocks and aborting one of the transactions to break the cycle.

36. What is a database cursor? How is it used?

A **cursor** in DBMS is a pointer to a result set of a query. It allows for row-by-row processing of query results, which is useful when dealing with large datasets.

Types of cursors:

- **Implicit cursors:** Automatically created by the DBMS for SELECT, INSERT, UPDATE, DELETE operations.
- **Explicit cursors:** Manually created by the programmer to process query results.

Example:

```
DECLARE cursor_example CURSOR FOR
SELECT * FROM Employee;
```

37. What are the different types of database locks?

There are several types of locks in DBMS to ensure data consistency when multiple transactions are involved:

- **Shared Lock (S Lock):** Allows multiple transactions to read a resource but prevents modification.
- **Exclusive Lock (X Lock):** Prevents any other transaction from reading or modifying the locked resource.
- **Intent Lock:** Signals that a transaction intends to lock a resource.
- **Update Lock (U Lock):** Used when a transaction intends to update a resource.

38. What is the difference between a clustered and non-clustered index?

- **Clustered Index:** Organizes the data in the table according to the index. There can only be one clustered index per table because the data rows can only be sorted one way.
- **Non-clustered Index:** Creates a separate structure from the table that holds pointers to the actual data rows. Multiple non-clustered indexes can be created on a table.

- **Intent Lock:** Signals that a transaction intends to lock a resource.
- **Update Lock (U Lock):** Used when a transaction intends to update a resource.

38. What is the difference between a clustered and non-clustered index?

- **Clustered Index:** Organizes the data in the table according to the index. There can only be one clustered index per table because the data rows can only be sorted one way.
- **Non-clustered Index:** Creates a separate structure from the table that holds pointers to the actual data rows. Multiple non-clustered indexes can be created on a table.

39. What is the importance of the COMMIT and ROLLBACK operations?

- **COMMIT:** Saves all changes made during the current transaction to the database permanently.
- **ROLLBACK:** Reverses all changes made during the current transaction, restoring the database to its previous state.
- Both operations ensure the **ACID** properties of transactions: **Atomicity** and **Durability**.

40. What is the difference between a superkey and a candidate key?

- **Superkey:** A set of one or more attributes that can uniquely identify a row in a table. It may contain unnecessary attributes.
- **Candidate Key:** A minimal superkey that uniquely identifies a row, with no redundant attributes. A table can have multiple candidate keys, and one is chosen as the **Primary Key**.

41. What are the different types of constraints in DBMS? Give examples.

Constraints in DBMS are rules applied to the data in a database to ensure its accuracy and integrity. The most common types of constraints are:

- **NOT NULL:** Ensures that a column cannot have NULL values. **Example:** `Name VARCHAR(50) NOT NULL;`
- **PRIMARY KEY:** Uniquely identifies each record in a table. It ensures that no duplicate rows exist and that no NULL values are allowed. **Example:** `ID INT PRIMARY KEY;`
- **FOREIGN KEY:** Ensures referential integrity between two tables by linking a column in one table to the primary key in another table. **Example:** `BranchCode INT FOREIGN KEY REFERENCES Branch(BranchCode);`
- **UNIQUE:** Ensures that all values in a column are distinct. Unlike the primary key, it allows NULL values. **Example:** `Email VARCHAR(100) UNIQUE;`
- **CHECK:** Ensures that values in a column satisfy a specific condition. **Example:** `Age INT CHECK (Age >= 18);`
- **DEFAULT:** Assigns a default value to a column if no value is provided during insertion. **Example:** `Status VARCHAR(10) DEFAULT 'Active';`

42. Explain the difference between a primary key and a unique key.

Primary Key:

- Uniquely identifies each record in a table.
- Cannot contain NULL values.
- A table can have only **one** primary key.

Unique Key:

- Ensures that all values in a column (or a set of columns) are unique across all rows.
- Can contain NULL values (unlike a primary key).
- A table can have **multiple** unique keys.

43. What is referential integrity in DBMS?

Referential Integrity ensures that relationships between tables are maintained correctly. It requires that the foreign key in one table must match a primary key or a unique key in another table (or be NULL). This ensures that data consistency is maintained, and there are no orphan records in the database.

Example: In the Orders table, if the CustomerID is a foreign key, it should match a valid CustomerID in the Customers table or be NULL.

44. How does DBMS handle concurrency control?

Concurrency control ensures that database transactions are executed in a way that prevents conflicts, such as data inconsistency, when multiple transactions are executed simultaneously. DBMS uses the following techniques:

- **Locking:** Transactions acquire locks (shared or exclusive) on the data to prevent other transactions from modifying it while one transaction is in progress. **Types of Locks:** Shared locks (S-lock) and exclusive locks (X-lock).
- **Timestamp Ordering:** Assigns a unique timestamp to each transaction and uses these

the Customers table or be NULL.

44. How does DBMS handle concurrency control?

Concurrency control ensures that database transactions are executed in a way that prevents conflicts, such as data inconsistency, when multiple transactions are executed simultaneously. DBMS uses the following techniques:

- **Locking:** Transactions acquire locks (shared or exclusive) on the data to prevent other transactions from modifying it while one transaction is in progress. **Types of Locks:** Shared locks (S-lock) and exclusive locks (X-lock).
- **Timestamp Ordering:** Assigns a unique timestamp to each transaction and uses these timestamps to determine the order in which transactions should be executed.
- **Optimistic Concurrency Control:** Transactions are executed without locking data, but before committing, the system checks whether there were conflicts with other transactions.
- **Two-Phase Locking:** Involves two phases—growing (acquiring locks) and shrinking (releasing locks)—to avoid deadlocks and ensure serializability.

45. What are the advantages of using foreign keys in DBMS?

- **Enforcing Data Integrity:** Ensures that the value of a foreign key matches a valid primary key or unique key, maintaining consistency.
- **Referential Integrity:** Prevents orphaned records in the database by enforcing valid relationships between tables.
- **Easy Data Maintenance:** Helps with cascading updates and deletions, meaning changes in the referenced table can automatically propagate to the referencing table (if configured with `ON UPDATE CASCADE` or `ON DELETE CASCADE`).
- **Improved Query Efficiency:** With foreign keys, database queries that join related tables are more efficient and meaningful.

46. What is a transaction log in DBMS?

A **transaction log** is a record that keeps track of all transactions executed on a database. It ensures that changes made by transactions are saved, and in case of a system failure, the log can be used to recover the database to its last consistent state. The transaction log contains:

- The details of each transaction (e.g., start, commit, rollback).
- Information about data modifications (insertions, updates, deletions).
- Details about the data before and after the change.

47. What is a materialized view in DBMS?

A **materialized view** is a database object that contains the results of a query. Unlike a regular view, which is a virtual table (it doesn't store data), a materialized view stores data physically, improving query performance by precomputing and storing results.

Use Case: Materialized views are commonly used for performance optimization in data warehousing and reporting systems, where the same data is frequently queried.

Example:

```
CREATE MATERIALIZED VIEW SalesSummary AS
SELECT Product, SUM(Quantity) FROM Sales GROUP BY Product;
```

48. What are the differences between an ER diagram and a relational schema?

Entity-Relationship Diagram (ERD):

- A **conceptual blueprint** that models entities, relationships, and attributes of the database. It visually represents the structure of the database.
- Used in the database design phase to understand how data entities relate to each other.

Relational Schema:

- A **logical schema** that defines the structure of a relational database, including tables, columns, relationships, and constraints.
- Represents how data is physically organized in tables with constraints such as primary keys, foreign keys, and data types.

49. What is the purpose of the GROUP BY clause in SQL?

The **GROUP BY** clause is used in SQL to group rows that have the same values in specified columns into summary rows, often with aggregate functions like `COUNT`, `SUM`, `AVG`, `MIN`, or `MAX`. It is typically used to organize data for reporting or analysis.

Example: This groups the employees by department and counts the number of employees in each department.

relationships, and constraints.

- Represents how data is physically organized in tables with constraints such as primary keys, foreign keys, and data types.

49. What is the purpose of the GROUP BY clause in SQL?

The **GROUP BY** clause is used in SQL to group rows that have the same values in specified columns into summary rows, often with aggregate functions like COUNT, SUM, AVG, MIN, or MAX. It is typically used to organize data for reporting or analysis.

Example: This groups the employees by department and counts the number of employees in each department.

```
SELECT Department, COUNT(*) FROM Employees
GROUP BY Department;
```

50. What are stored functions in DBMS?

A **stored function** is a set of SQL statements that can be executed in the database. It accepts input parameters, performs some logic, and returns a value. Stored functions are similar to stored procedures but differ in that they must return a value.

Example:

```
CREATE FUNCTION GetEmployeeSalary(EmployeeID INT)
RETURNS DECIMAL(10,2)
BEGIN
    DECLARE salary DECIMAL(10,2);
    SELECT Salary INTO salary FROM Employee WHERE ID = EmployeeID;
    RETURN salary;
END;
```

Advanced DBMS Interview Questions

DBMS Advanced Interview Questions focus on in-depth knowledge and problem-solving skills for complex database scenarios. This category evaluates your ability to handle large-scale data, optimize query performance, and understand advanced DBMS topics like distributed databases, indexing strategies, and concurrency control mechanisms. Being proficient in these areas is crucial for roles involving high-performance databases and large-scale systems.

51. What are the various types of normalization techniques? Explain with examples.

Normalization is the process of organizing data in a database to eliminate redundancy and improve data integrity. The primary goal is to minimize the chances of anomalies when performing operations like insertions, deletions, and updates. There are several levels or **normal forms** in normalization:

1. **First Normal Form (1NF):** Ensures that the data in the table is atomic, meaning each column contains indivisible values (no multi-valued attributes).

Example:

```
Student (ID, Name, Subjects)
```

2. **Second Normal Form (2NF):** 2NF is achieved when the table is in 1NF and all non-key attributes are fully dependent on the primary key.

Example: Consider the following table

```
StudentCourse (StudentID, CourseID, InstructorName)
```

Here, InstructorName depends on CourseID, not on StudentID. To make it 2NF, we separate the data:

```
Student (StudentID, Name)
Course (CourseID, InstructorName)
Enrollment (StudentID, CourseID)
```

3. **Third Normal Form (3NF):** A table is in 3NF if it is in 2NF and there is no transitive dependency (non-key attributes depend on other non-key attributes).

Example:

```
Employee (EmpID, EmpName, DeptID, DeptName)
```

Here, DeptName depends on DeptID. To bring this into 3NF:

```
Employee (EmpID, EmpName, DeptID)
Department (DeptID, DeptName)
```


51. What are the various types of normalization techniques? Explain with examples.

Normalization is the process of organizing data in a database to eliminate redundancy and improve data integrity. The primary goal is to minimize the chances of anomalies when performing operations like insertions, deletions, and updates. There are several levels or **normal forms** in normalization:

1. First Normal Form (1NF): Ensures that the data in the table is atomic, meaning each column contains indivisible values (no multi-valued attributes).

Example:

```
Student (ID, Name, Subjects)
```

2. Second Normal Form (2NF): 2NF is achieved when the table is in 1NF and all non-key attributes are fully dependent on the primary key.

Example: Consider the following table

```
StudentCourse (StudentID, CourseID, InstructorName)
```

Here, InstructorName depends on CourseID, not on StudentID. To make it 2NF, we separate the data:

```
Student (StudentID, Name)
Course (CourseID, InstructorName)
Enrollment (StudentID, CourseID)
```

3. Third Normal Form (3NF): A table is in 3NF if it is in 2NF and there is no transitive dependency (non-key attributes depend on other non-key attributes).

Example:

```
Employee (EmpID, EmpName, DeptID, DeptName)
```

Here, DeptName depends on DeptID. To bring this into 3NF:

```
Employee (EmpID, EmpName, DeptID)
Department (DeptID, DeptName)
```

4. Boyce-Codd Normal Form (BCNF): A stricter version of 3NF where every determinant is a candidate key.

Example:

```
Student (StudentID, CourseID, InstructorID)
```

And the constraint is such that InstructorID determines CourseID, it would not be in BCNF because InstructorID is not a candidate key. To achieve BCNF:

```
Instructor (InstructorID, CourseID)
Enrollment (StudentID, CourseID)
```

52. What are the different phases of the DBMS query processing cycle?

The DBMS query processing cycle consists of several phases that transform the high-level query (SQL) into executable actions:

- 1. Parsing:** The SQL query is parsed to check its syntax and semantics. The DBMS ensures that the query is valid according to the SQL syntax and the database schema.
- 2. Translation:** The query is translated into an internal form, such as a relational algebra expression or an execution plan.
- 3. Optimization:** The DBMS optimizes the query to determine the most efficient execution plan, considering factors like indexes, joins, and available resources.
- 4. Execution:** The optimized query plan is executed by the query processor, which accesses the data from the database and returns the results.

53. What are the different types of backups in DBMS?

There are several types of backups in DBMS:

- **Full Backup:** A full backup copies the entire database, including all data and the database structure. It is the most comprehensive but can take up a lot of storage and time.
- **Incremental Backup:** An incremental backup only copies the data that has changed since the last backup (either full or incremental). This saves space and time but requires the restoration of the full backup and all incremental backups.

4. **Execution:** The optimized query plan is executed by the query processor, which accesses the data from the database and returns the results.

53. What are the different types of backups in DBMS?

There are several types of backups in DBMS:

- **Full Backup:** A full backup copies the entire database, including all data and the database structure. It is the most comprehensive but can take up a lot of storage and time.
- **Incremental Backup:** An incremental backup only copies the data that has changed since the last backup (either full or incremental). This saves space and time but requires the restoration of the full backup and all incremental backups.
- **Differential Backup:** A differential backup copies all changes made since the last full backup. It's faster than a full backup and simpler to restore than incremental backups.
- **Transaction Log Backup:** A transaction log backup copies the transaction log, which records all transactions performed on the database. This allows for point-in-time recovery.

54. What is the use of the "WITH CHECK OPTION" in SQL views?

The "WITH CHECK OPTION" is used when creating a view in SQL to ensure that any insert or update operation on the view must adhere to the conditions defined in the view's WHERE clause. If the inserted or updated data violates these conditions, the operation will be rejected.

Example: Here, if a user tries to insert or update a Student record with a status other than 'Active', the operation will fail.

```
CREATE VIEW ActiveStudents AS
SELECT * FROM Students WHERE Status = 'Active'
WITH CHECK OPTION;
```

55. Explain the concept of a B-tree and B+ tree in DBMS.

B-tree (Balanced Tree):

- A **B-tree** is a self-balancing tree data structure that maintains sorted data and allows searches, insertions, deletions in logarithmic time.
- B-trees are used in databases and file systems to store large amounts of data. All nodes in a B-tree can have multiple children, which increases the efficiency of searching.
- Stores data in both internal and leaf nodes.

B+ tree:

- A **B+ tree** is an extension of the B-tree and is widely used in databases for indexing. It differs in that it has a linked list at the leaf level and stores all records in the leaf nodes.
- The internal nodes of the B+ tree store only keys and pointers to the next level of the tree, while the leaf nodes contain actual data or pointers to the data.
- Stores data only in the leaf nodes and uses the internal nodes for indexing.

56. What is a hashing technique in DBMS? How does it work?

A **hashing technique** in DBMS is used to map data (such as a key) to a fixed-size value or address, using a hash function. It is primarily used for quick data retrieval, particularly in hash indexes or hash tables. **Example:** A hash function might map a StudentID of 123 to a bucket index of 3. The student's record would be stored in the corresponding bucket.

How it works:

1. A hash function takes the key (e.g., a record's ID) and calculates a **hash value**.
2. This hash value determines the **bucket** or **slot** where the data is stored.
3. When searching for a record, the hash function is applied again to the key to find the corresponding bucket.

57. What is the difference between a trigger and a stored procedure?

Trigger:

- A **trigger** is an automatic action executed by the DBMS when a specific event occurs on a table, such as an INSERT, UPDATE, or DELETE.
- It cannot be invoked manually and is tied to a specific event.

Stored Procedure:

- A **stored procedure** is a precompiled set of SQL statements that can be executed explicitly by a user or application.
- It is invoked manually, and it can accept input parameters and return output.

58. Explain the concept of data partitioning in DBMS.

7:51



VoLTE

3.11 KB/s



5G



 69%

Databases

SQL

MySQL

PostgreSQL

PL/SQL

MongoDB

SQL Cheat Sheet

SQL Interview Questions

MySQL Interv

Sign In

3. When searching for a record, the hash function is applied again to the key to find the corresponding bucket.

57. What is the difference between a trigger and a stored procedure?

Trigger:

- A **trigger** is an automatic action executed by the DBMS when a specific event occurs on a table, such as an INSERT, UPDATE, or DELETE.
- It cannot be invoked manually and is tied to a specific event.

Stored Procedure:

- A **stored procedure** is a precompiled set of SQL statements that can be executed explicitly by a user or application.
- It is invoked manually, and it can accept input parameters and return output.

58. Explain the concept of data partitioning in DBMS.

Data partitioning is the process of dividing large datasets into smaller, more manageable segments (partitions) to improve performance, scalability, and availability. Each partition can be stored and processed separately.

Types of partitioning:

1. **Horizontal Partitioning:** Divides data by rows. For example, splitting data by time range (e.g., 2020 data in one partition, 2021 in another).
2. **Vertical Partitioning:** Divides data by columns. For example, putting frequently accessed columns in one partition and less frequently accessed columns in another.
3. **Range Partitioning:** Data is divided based on a range of values (e.g., age groups, date ranges).
4. **Hash Partitioning:** Data is distributed across partitions based on a hash value derived from a key column.

59. What is the role of the DBMS in handling data integrity and security?

The DBMS plays a critical role in ensuring:

- **Data Integrity:** Through constraints like **Primary Keys**, **Foreign Keys**, and **Check Constraints**, the DBMS ensures data consistency and accuracy.
- **Data Security:** DBMS systems provide user authentication, access control, and encryption mechanisms to protect data from unauthorized access and breaches. It also supports role-based access control (RBAC), ensuring that only authorized users can perform certain actions on the data.

60. How is DBMS different from a file-based system?

Aspect	DBMS	File-based System
Data Organization	Data is stored in tables (relations) with structured schemas, supporting complex queries and relationships.	Data is stored in flat files without any relationship between data.
Data Redundancy	Minimizes redundancy through normalization.	Data redundancy is common as data may be duplicated across different files.
Data Integrity	Ensures data integrity through constraints (e.g., primary keys, foreign keys, and check constraints).	Data integrity is hard to enforce, as files don't have built-in integrity checks.
Security	Provides advanced security features like user authentication, access control, and encryption.	Security is managed at the file system level, which is less robust than DBMS security features.
Concurrency Control	Handles concurrent access using locking mechanisms, ensuring data consistency.	No built-in concurrency control; data corruption may occur when multiple users access the same file.
Data Access	Supports complex querying and transaction management (e.g., SQL).	Data access is limited to basic file operations like reading, writing, and appending.
Backup and Recovery	Automatic backup and recovery mechanisms, often integrated into the system.	Backup and recovery mechanisms are manual and may not be as robust.

Open In App

Early scalable to handle large datasets

Not as scalable managing large datasets

60. How is DBMS different from a file-based system?

Aspect	DBMS	File-based System
Data Organization	Data is stored in tables (relations) with structured schemas, supporting complex queries and relationships.	Data is stored in flat files without any relationship between data.
Data Redundancy	Minimizes redundancy through normalization.	Data redundancy is common as data may be duplicated across different files.
Data Integrity	Ensures data integrity through constraints (e.g., primary keys, foreign keys, and check constraints).	Data integrity is hard to enforce, as files don't have built-in integrity checks.
Security	Provides advanced security features like user authentication, access control, and encryption.	Security is managed at the file system level, which is less robust than DBMS security features.
Concurrency Control	Handles concurrent access using locking mechanisms, ensuring data consistency.	No built-in concurrency control; data corruption may occur when multiple users access the same file.
Data Access	Supports complex querying and transaction management (e.g., SQL).	Data access is limited to basic file operations like reading, writing, and appending.
Backup and Recovery	Automatic backup and recovery mechanisms, often integrated into the system.	Backup and recovery mechanisms are manual and may not be as robust.
Scalability	Easily scalable to handle large datasets and multiple users.	Not as scalable; managing large amounts of data and multiple users is difficult.
Maintenance	Centralized maintenance, with tools for optimization, tuning, and troubleshooting.	Maintenance is usually handled at the file system level, which may require manual intervention.
Transaction Management	Supports ACID (Atomicity, Consistency, Isolation, Durability) properties for reliable transaction management.	No built-in support for transactions or ACID properties, making it prone to errors during data modification.

Conclusion

Preparing for a **DBMS interview** requires a strong understanding of both fundamental and advanced concepts. From basic database operations to handling large-scale data with advanced indexing strategies, mastering these DBMS interview questions will prepare you for a variety of challenges you may encounter in your career. By practicing these questions, you'll enhance your problem-solving skills, boost your confidence, and be ready to tackle any database-related challenges in your interview.

 Comment

More info 

Advertise with us

Next Article 

Commonly asked DBMS Interview Questions | Set 2

Similar Reads

1. Top 50 Database Interview Questions and Answers for 2025

2. Top 40+ Database Testing Interview Questions 2025 (With Answers)

3. Top HR Interview Questions and Answers (2025)

4. Top Infosys Interview Questions and Answers For 2024

5. Top 50 SQL Questions For Data Analyst Interview

6. Top 60+ Data Engineer Interview Questions and Answers

7. 30 SQL Interview Questions For Business Analyst in 2025

8. Top 50 Data Engineering Interview Questions and Answers

9. Top 50 SQLite Interview Questions for 2024

10. Top 80+ Data Analyst Interview Questions and Answers